

SpliTEE: Improving LLM Inference on Trusted Hardware with Differentially Private GPU Outsourcing

Shashie Dilhara Batan Arachchige, Robin Carpentier, Hassan Jameel Asghar, Dali Kaafar
School of Computing, Macquarie University

{shashiedilhara.batanarachchige, robin.carpentier, hassan.asghar, dali.kaafar}@mq.edu.au

September 15, 2026

Abstract

User prompts provided to large language models (LLMs) may contain sensitive or private information that can potentially be misused by these remotely deployed models, such as through inadvertent memorization during retraining. One way to protect user prompts is to execute the LLM inside a trusted execution environment (TEE), with the guarantee that the service provider has no access to computations performed within or the information exchanged with the TEE. However, this results in slow inference times as current TEEs are primarily CPU-based and significantly slower than GPUs, which are optimized for LLM inference. To circumvent this, Tramèr and Boneh (2019) proposed a system called Slalom which splits neural network inference between a TEE and the untrusted GPU. They encrypt intermediate inputs to components outsourced to the GPU to prevent them from leaking information about the prompt. In this paper, we extend this split-inference architecture to LLM inference and instead protect intermediate inputs using differential privacy. We first demonstrate that *masking* intermediate representations is necessary by showing that a prompt-reconstruction attack can recover prompts from these representations with nearly 80% accuracy. Our main contribution is a global sensitivity analysis of key functions in LLM inference, which bounds the required scale of differentially private noise. Unlike encryption, differential privacy avoids quantization, allowing the LLM to remain in the floating-point domain. We also derive an upper bound on the floating-point error from masking and subsequent noise cancellation in the TEE as a function of the privacy parameter ϵ , which can be used to ensure that the quality of the LLM response is not affected. We implement our architecture using the Intel TDX TEE and evaluate it with two LLMs: **Llama-3.2-3B** and **Qwen3-4B**. Our split execution is nearly twice as fast as fully CPU-based inference inside TDX. Moreover, it is 5–15 seconds faster than encryption-based Slalom while also achieving higher accuracy. Finally, we demonstrate that prompt reconstruction, even with knowledge of the differential privacy mechanism, cannot recover more information than is contained in an unrelated prompt.

1 Introduction

The use of artificial intelligence has proliferated in the last couple of years due to the ever-expanding and improving capabilities of large language models (LLMs). At the same time there are growing privacy concerns due to the remote deployment of these LLMs. Users submit their requests to these LLMs using prompts that may contain sensitive information about individuals and businesses. Naturally, one would not want this information to be misused. For instance, it is routine practice to use user prompts and subsequent responses to train future versions of the LLM. As a result, the sensitive information contained in these prompts can be memorized by LLMs [20]. One option available to the user is to opt-out of his/her data being used to retrain the model.¹ However, there are a number of issues with this solution. This generally does not apply to data already submitted before toggling the feature, by default the opt-out feature may be unchecked, and even if the user opts out, some data or metadata may still be used for training [23]. Another option on the other extreme is to deploy LLMs locally on the user’s device, but this is infeasible due to their ever increasing

¹For example, by turning off the “Improve the model for everyone” option in ChatGPT (<https://chatgpt.com/>).

sizes. Besides, this deployment model is not suitable for LLM service providers who want to protect their intellectual property, the learned weights of the model.

Technical solutions to this problem include making the prompt more private by using differential privacy [12] or carrying out LLM inference in the encrypted domain using homomorphic encryption or secure multiparty computation. A differentially private solution probabilistically replaces either individual words or sentences in the prompt with semantically similar counterparts (see, for example [21]). This is not ideal for utility as it perturbs words in the prompt, and depending on the task, the user may want to keep the text in the prompt unchanged, e.g., specific sales numbers. Cryptographic solutions through homomorphic encryption and secure multiparty computations are computationally or communication-wise expensive, require changes to the LLM to make it more suitable for cryptographic operations, and some part of the LLM computation is assumed to be done publicly, e.g., by the client. See these excellent surveys for the state-of-the-art in this domain [1, 2]. If these challenges are overcome, these solutions are promising as they do not alter the prompt.

Another solution, which is also related to the theme of this paper, is to execute the LLM inside a trusted execution environment (TEE). The service provider hosts the LLM within the TEE, with the guarantee that computations performed inside the TEE can neither be observed nor be tampered with by the service provider. These are standard assumptions for computations performed within a TEE.² Unfortunately, end-to-end inference within the TEE is slow, as current TEEs are primarily CPU-based which are significantly slower than GPUs for LLM inference. We also demonstrate this performance gap in this paper. While the first commercial GPU-TEE was announced recently, namely NVIDIA Confidential Computing [9], it is only available on one commercial GPU and specifications are scarce, which renders the security analysis of the system difficult [16]. Nevertheless even GPU-based TEEs are likely to be slower than state-of-the-art GPUs outside the trusted zone.

To solve this issue, researchers have proposed a split-inference architecture, where the expensive operations during LLM inference, namely matrix multiplications, are offloaded or outsourced to the faster but untrusted GPU, and the rest of the components are kept inside the trusted TEE [34].³ To prevent information leakage, the inputs to the offloaded components are *masked* by additively blinding them with random nonces. The GPU performs the multiplication as if it received the real input. Due to the linearity of matrix multiplications, the original matrix product can be retrieved from the result returned by the GPU by subtracting the randomized component. Moreover, this can be partly preprocessed, and hence expensive operations are not done at the slower TEE. An issue with the approach used in [34] is that the matrix product needs to be *quantized* to work in finite fields, which results in accuracy loss.

In this paper we look at the LLM split-inference architecture from another angle: instead of encrypting the outsourced inputs via a stream cipher as is done in Slalom, the name of the system proposed in [34], we add differentially private noise calibrated to the sensitivity of the operation. The advantage is that we remain in the original floating-point arithmetic domain in which the LLM is implemented. Since our goal is prompt privacy, differential privacy is a reasonable notion to apply in the split-inference setting. However, as we show, we cannot add unbounded differentially private noise without running into floating-point error accumulation issues. The good news is that we can tune the privacy parameter ϵ so that error never passes the threshold to impact accuracy. Our method is faster than Slalom, and also incurs no accuracy penalty depending on the privacy level ϵ . Figure 1 shows the high-level overview of our scheme.

To summarize, we make the following contributions:

1. We present a split-architecture, where LLM inference is split between a trusted but slower TEE, and a faster but honest-but-curious GPU. The linear operations are outsourced to the GPU as is done in [34], but we mask the inputs to the GPU using differentially private noise so that the GPU cannot infer the user prompt. The GPU being honest-but-curious is a realistic assumption in many use cases as the service provider is rightly concerned about maintaining its reputation.
2. Using the open source **Llama-3.2-3B-Instruct** LLM as the base, we identify common components in LLMs that can be algebraically outsourced. As the inputs to these components are outputs of the preceding components, we derive the global sensitivity bounds of these functions, which measure the

²Of course, TEEs are no panacea. It may be possible to infer some information about user prompts through side channel attacks and metadata exchanged between the TEE and the untrusted part of the host machine [7].

³Although the scheme in [34] is built for deep neural networks, it can be adapted for LLM inference, as is done in [39].

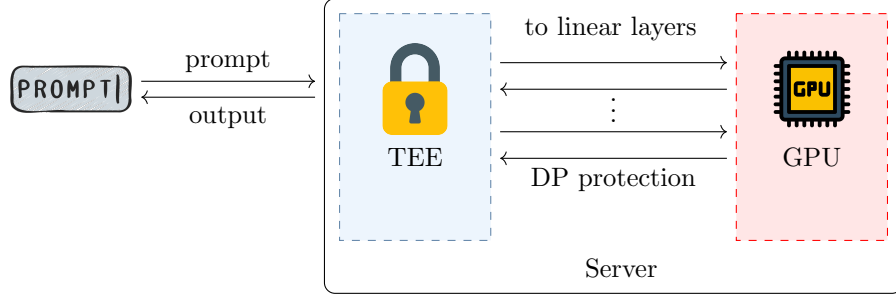


Figure 1: Overview of the LLM deployment architecture. The TEE is trusted whereas the GPU is semi-trusted, i.e., it follows the protocol but may want to infer more information about the user prompt.

required scale of differentially private noise [13]. Our global sensitivity analysis is a contribution in itself and can be used in any system that outsources part of LLM inference to another entity without disclosing intermediate function inputs and outputs.

3. We show that removing differentially private noise from the result computed by the GPU on the outsourced computation results in accumulation of floating-point errors that can negatively impact accuracy of the LLM response. We derive an upper bound on this error for linear operations, and show that it closely matches real execution error. Building on this, we show that the value of the privacy parameter ϵ can be tweaked so that the error bound remains below a threshold to ensure accuracy is not impacted.
4. We implement our scheme on a machine equipped with Intel Trust Domain Extensions (TDX) and a GPU. Using both the *Llama-3.2-3B-Instruct* and *Qwen3-4B-Instruct* LLMs, we show that end-to-end CPU-based inference on the protected TDX virtual machine is almost two times slower than our scheme. We further show that our scheme is between 5 to 15 seconds faster than Slalom [34] adapted to LLMs.
5. We perform a prompt reconstruction attack by training an LLM model with the outsourced inputs. The attack shows that adding differentially private noise is essential, as the prompt reconstruction attack can reconstruct the prompt with almost 80% accuracy if the inputs are *unmasked*. We further show that the same attack, even when trained on auxiliary noisy inputs, is not able to infer the user prompt significantly better than guessing random tokens expected to be common between unrelated prompts. Our implementation is publicly available.⁴

2 Related Work

Slalom [34] is one of the earliest approaches to combine a TEE with an untrusted GPU for private and verifiable neural-network inference. Instead of executing the complete model inside the TEE, it outsources expensive linear operations while masking the operator input with a single-use random mask. The corresponding correction is precomputed inside the TEE so that the original linear result can be recovered after GPU execution. Slalom also uses Freivalds’ algorithm [14] to verify the correctness of the outsourced computation. Our approach follows a similar high-level idea of protecting outsourced linear computation, but differs in both the masking mechanism and execution setting. Slalom uses pseudorandom masks over finite-field representations, whereas our approach adds Gaussian noise to floating-point activation tensors based on the sensitivity of each outsourced operator. Slalom was designed for conventional Deep Neural Network (DNN) workloads and evaluated using Intel SGX, while our work considers Transformer-based LLM inference inside an Intel TDX virtual machine. We further show that our system is better at maintaining the accuracy of LLM responses, as we do not quantize the model as needs to be done through Slalom-based LLM inference.

⁴<https://github.com/DPVault/SpliTEE>

While Slalom focuses on private and verifiable outsourcing of linear computation, other TEE-based systems consider different protection goals. SOTER [31] also partitions neural-network inference between a TEE and an untrusted accelerator to reduce the cost of executing the complete model inside trusted hardware. Its main objective, however, is to protect the intellectual property of the model when inference is performed on a user device. SOTER identifies *associative* operators and transforms their *parameters* by multiplying them with a random scalar mask before outsourcing the operator to the untrusted GPU; the intended result is then recovered inside the TEE using the corresponding inverse scaling. This differs from our setting mainly in the privacy objective. It protects model parameters from an untrusted device owner, whereas our work focuses on protecting prompt information and intermediate activations exposed to an untrusted GPU during LLM inference. For instance, while we could adapt their technique to multiply intermediate inputs with a vector of random scalars, this will exacerbate the floating-point issues discussed in our paper. Our approach also targets Transformer-based LLM execution, while SOTER considers more general neural-network models.

Transformer-specific outsourcing introduces a broader set of operations that must be protected. TwinShield [39] addresses confidential execution of transformer models in an untrusted cloud environment. It aims to protect model parameters, input data, and computation integrity while reducing the amount of computation performed inside the TEE. Unlike earlier approaches that primarily outsource linear operations, TwinShield extends outsourcing to transformer-specific operations, attention matrix multiplication and the non-linear softmax function. It also provides a verification mechanism for the outsourced computation. While their mechanism to mask inputs to the linear layers is the same as Slalom, through additive random masks, the non-linear components are protected through a combination of scalar masking and permutation. The former can be proven to be secure assuming the security of the underlying cipher, but the latter is ad hoc and susceptible to algebraic attacks. Moreover, their scheme suffers from the same accuracy issues due to model quantization.

Beyond the design of individual partitioning mechanisms, recent work has also examined whether exposing part of a model outside the TEE can itself create a security risk. Zhang et al. [41] study the security of TEE-shielded DNN partitioning for on-device inference. They show that existing partitioning approaches can remain vulnerable to model stealing and membership inference when information from the untrusted portion of the model is combined with public model knowledge. To address this, they propose TEESlice, which follows a partition-before-training strategy so that the weights placed outside the TEE are not trained on private data. The architecture combines a public pre-trained backbone with privacy-sensitive model slices retained inside the TEE, while the input to outsourced computation is protected using a one-time masking mechanism similar in spirit to Slalom. Clearly, this solution protects leakage of model parameters and training data to an untrusted device owner, and not user prompts.

Intermediate representations have also been shown to leak information about model inputs through inversion and reconstruction attacks. Song and Raghunathan [32] showed that embeddings can retain enough information to recover words from the original input. Later works extended this to full-sentence reconstruction using optimization-based attacks [27] and learning-based approaches that train a generative decoder to recover the input [24]. The latter particularly motivates our reconstruction attacker. Dong et al. [11] further study inversion of hidden states exposed during split LLM inference and show that adding Laplace noise directly to these states can reduce reconstruction only at a substantial utility cost. This differs from our setting because their noisy representation is also used for inference. In our design, noise is added only to the outsourced operator input and the corresponding correction is removed inside the TEE before inference continues. Thus, the attacker observes the masked representation, while the model proceeds using the recovered linear result, subject only to numerical error from masking and correction.

3 Background

3.1 Trusted Execution Environment (TEE)

Briefly a trusted execution environment (TEE) is a secure area of a processor which provides privacy and integrity of its data and computation by isolating it from any outside entities including the operating system and other applications running on the machine. Furthermore, the user can verify the code executed at the TEE is correct and not tampered with, through *remote attestation*. The TEE achieves this by sending

the cryptographic hash of its application code, cryptographically signed by the hardware-protected private attestation key of the vendor (e.g., Intel TDX). The client can verify the signature and the hash via the corresponding public verification key through the vendor’s certificate [26].

3.2 Components of an LLM

In order to identify which components of an LLM can be efficiently outsourced to the GPU while masking their inputs, we detail the general architecture of an LLM. Of course, LLMs are not homogeneous, and vary in their architecture.⁵ Our goal is to identify components common in most if not all LLMs. We use the open source **Llama-3.2-3B-Instruct** model,⁶ henceforth only referred to as **Llama-3B**, as the reference model. Minor differences with other models are discussed in Section 3.2.6.

3.2.1 Overview and Notations

Most LLMs are based on the transformer architecture [37], which divides the computation into the following components:

1. An Embedding Model
2. A Transformer Block (repeated a fixed number of times), itself containing
 - (a) A Multi-Head Self-Attention mechanism
 - (b) A Feed-Forward Network
3. A Language Modeling Head

The computation begins by processing all tokens in the prompt at once, i.e., as one matrix (explanation to follow). Once a token has been generated after Step 3, it becomes part of the input to generate the next token. After this, Steps 2 and 3 are repeated until the number of desired output tokens have been produced. Below, we give more details of each of the components mentioned above. A pictorial representation of these components is shown in Figure 2.

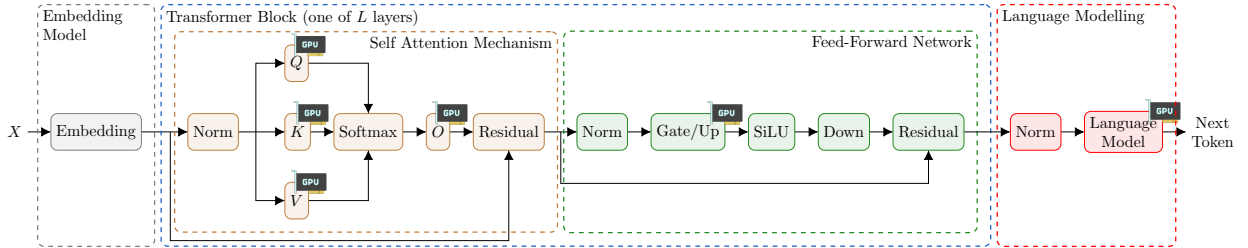


Figure 2: A breakdown of LLM components based on Llama-3B. The components offloaded to the GPU are marked with the icon .

3.2.2 Embedding Model

The embedding model associates each token of the prompt with an embedding vector of dimension d . Thus the input prompt can be thought of as the token matrix X , which is composed of n tokens $(\mathbf{x}_1, \dots, \mathbf{x}_n)$, where each token is a vector in $\mathbb{R}^d$ of embedding dimension d . The dimension d is also called the hidden size. The dimension is $d = 3072$ in Llama-3B.

⁵ #NotAllLLMs

⁶ See <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>

3.2.3 Transformer Block – Self Attention Mechanism

As illustrated in Figure 2, the first part of the transformer block is the multi-head self-attention mechanism. Roughly, this mechanism calculates how much each token should “attend” to every previous token to better understand its context. The self-attention mechanism has the following sub-components.

Input Layer Normalization. In Llama-3B, the attention mechanism starts with the normalization layer (called, the pre-norm transformer). Each token embedding $\mathbf{x}$ is normalized independently using root-mean square normalization (RMSNorm) (see Definition 5). This normalization is done at the start of each layer (see Figure 2). Each layer has its own learnable parameters $\alpha, \beta \in \mathbb{R}^d$ (see Eq. (5)). After this step, the initial token matrix X is transformed to the normalized token matrix $\hat{X}$.

Linear Projections (Q, K, V). Three independent linear projections of the input are computed using (learned) weight matrices $W_{\text{qry}}, W_{\text{key}}$ and W_{val} resulting in the quantities $Q = W_{\text{qry}}\hat{X}$, $K = W_{\text{key}}\hat{X}$ and $V = W_{\text{val}}\hat{X}$. The notation Q, K , and V stand for query, key, and value, respectively. In Llama-3B, the matrices $W_{\text{qry}}, W_{\text{key}}$ and W_{val} have dimensions $d_{\text{qry}} \times d, d_{\text{key}} \times d$ and $d_{\text{val}} \times d$ respectively, where for example $d_{\text{qry}} = d_{\text{key}} = d_{\text{val}} = 500$. We shall assume that they are equal, and represent the variable dimension by d_{key} . Thus, matrices Q, K and V have dimensions $d_{\text{key}} \times n$ each.⁷

Rotary Positional Embeddings (RoPE). The attention mechanism does not know the order of the tokens in the prompt. This positional information is injected through rotary positional embeddings into Q and K . With this information, the model can tell whether a token is positioned earlier or later in the sequence.

Scaled Dot-Product Attention. This component combines the matrices Q, K and V as

$$H = \text{softmax} \left(\frac{Q^T K}{\sqrt{d_{\text{key}}}} \right) V^T$$

Note that $Q^T K$ is a matrix of dimensions $n \times n$, since Q^T has dimension $n \times d_{\text{qry}}$ and K has dimension $d_{\text{key}} \times n$, and we assume $d_{\text{qry}} = d_{\text{key}}$. The output of the softmax function is thus also an $n \times n$ matrix. This is then multiplied by the $n \times d_{\text{key}}$ matrix V^T , to produce an $n \times d_{\text{key}}$ matrix H .

Linear Projection (O). Prior to this component, the outputs from each of the h attention heads are combined together, resulting in the final matrix H being of size $n \times h d_{\text{val}} = n \times d$, as h is set as d/d_{key} .⁸ The next component is another linear projection using the (learned) weight matrix $W_{\text{oh}} \in \mathbb{R}^{d \times d}$ as

$$O = W_{\text{oh}} H^T$$

which results in a $d \times n$ matrix O .

Residual Addition. At this point the original (unnormalized) input X is added to O to obtain

$$Y = O + X$$

which is again a $d \times n$ matrix, which is the final output of the attention mechanism.

3.2.4 Transformer Block – Feed Forward Neural Network (MLP)

This component has the following subcomponents.

⁷PyTorch stores the weight matrices transposed, and then transposes them at runtime. See <https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html>.

⁸See https://huggingface.co/docs/transformers/model_doc/llama.

Post Attention Layer Normalization. This is the same operation as input layer normalization from the attention mechanism except that the learned parameters are different (see Eq. (5)). The matrix Y is thus transformed into the matrix $\hat{Y}$ with RMS-normalized entries.

Linear Projections (*Gate*, *Up*). Two independent projections of the input are computed, using the (learned) weight matrices $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d_{\text{ff}} \times d}$, where d_{ff} is the model’s feed-forward dimension, e.g., 8,192. This gives rise to the $d_{\text{ff}} \times n$ matrices

$$G = W_{\text{gate}}\hat{Y} \text{ and } U = W_{\text{up}}\hat{Y}$$

Activation Function. In Llama-3B, the activation function used is the Sigmoid Linear Unit (SiLU).⁹ We denote this function by sig , which can be applied element-wise on a matrix (see Eq. (9) for the definition). After application of the activation function, we obtain the resulting matrix B as

$$B = U \odot G \odot \text{sig}(G)$$

where $\odot$ denotes element-wise matrix product. Note that since each matrix is of dimension $d_{\text{ff}} \times n$, the matrix B is also of the same dimension.

Linear Projection (*Down*). The final part of this component is a linear projection using the (learned) weight matrix $W_{\text{down}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ giving us

$$D = W_{\text{down}}B$$

which is a $d \times n$ matrix.

Residual Addition This is where the residual output from the self-attention mechanism, i.e., Y , is added to D , and thus D is updated as $D \leftarrow D + Y$.

3.2.5 Language Modelling Head

The transformer block described in Sections 3.2.3 and 3.2.4 is run L times. Here L represents the number of attention blocks, also called layers, e.g., 16. The output after each layer is called a *hidden state*. The output after the L th layer is called the *last hidden state*. This last hidden state is given to the language modelling head to generate a token. The main tasks involved in this component are as follows.

Final Layer Normalization. This is again the same operation as in input layer normalization except that the learned parameters are different.

Language Modelling. For next token prediction, we only consider the last vector of the last hidden state, i.e., corresponding to the last token. This component uses the model’s token vocabulary, denoted vocab . The vocabulary’s size differs according to the model. In Llama-3B, it is 128,256.¹⁰ Each token in the vocabulary has an associated token embedding in $\mathbb{R}^d$. This layer then performs a dot product of the last hidden vector with the token embedding of each token in the vocabulary. This is essentially a multiplication with the matrix of token embeddings of all tokens in the vocabulary, which we call W_{lm} , whose weights are again obtained through training. We get a vector of values, one for each token in the vocabulary. The model then applies the softmax function to get a vector of probabilities. Finally, the next token is chosen based on these probability vectors and a configuration, e.g., always choosing the highest probable token is called greedy decoding.

⁹See <https://docs.pytorch.org/docs/stable/generated/torch.nn.SiLU.html>

¹⁰See <https://huggingface.co/HenryShan/Llama-3.2-3B-GSM8K-CoT>.

3.2.6 Differences From Other LLMs

The architecture described above is based primarily on **Llama-3B**, which is also the main model used in our evaluation. However, the proposed outsourcing approach is not intended to depend on a single model architecture. To examine how it can be applied more broadly, we consider other widely used Transformer-based model families and identify architectural differences that are relevant to the outsourced computation.

In particular, we consider Qwen, which is also included in our experimental evaluation, and discuss Gemma and Mistral to examine how the approach extends to other modern LLM families. Since our approach outsources the computationally expensive linear projections while keeping the other model operations within the TEE, the main question is whether these models retain similar linear operations and how the surrounding computations differ.

We first consider the Qwen family. Qwen models follow a similar decoder-only Transformer architecture to Llama, but some design choices differ between model generations. For example, **Qwen2.5-3B** uses bias terms in the Q , K , and V projections, whereas **Qwen3-4B** uses bias-free projections. Here, a bias is an additional learned value that helps the projection adjust its output beyond what is produced by the matrix multiplication alone. For a biased linear layer with output $W\mathbf{x} + \mathbf{b}$, where $\mathbf{b}$ is the bias vector, outsourcing the masked input produces $W(\mathbf{x} + \boldsymbol{\eta}) + \mathbf{b}$. Subtracting the masking correction $W\boldsymbol{\eta}$ inside the TEE recovers $W\mathbf{x} + \mathbf{b}$, so no additional correction is required for the bias term. Qwen3 also applies RMSNorm separately to the Q and K representations after their respective projections. This additional normalization remains inside the TEE and does not change the outsourced linear projections themselves.

We next look at the Gemma family¹¹. For this comparison, we use **Gemma 3** [33]. It retains the same main linear projections in the attention and feed-forward blocks, but introduces several differences in the surrounding operations. Gemma 3 applies additional RMSNorm operations, also normalizes the Q and K representations after projection. It combines local sliding-window attention with global attention [33]. Sliding-window attention limits each token to attending within a nearby region, while global attention allows information to be shared across the full sequence. Its feed-forward network also uses a GELU-based gated activation instead of SiLU. These differences affect how the projection outputs are processed, but the main attention and feed-forward projections remain linear.

Finally, we consider Mistral¹² [22]. For this comparison, we inspect **Mistral-7B-Instruct-v0.3**. This model uses bias-free linear projections, follows the same general RMSNorm placement as **Llama-3B**, and uses a SiLU-based gated feed-forward block. It also does not apply additional normalization to the query or key representations. Unlike the original Mistral 7B release [22], this does not use sliding-window attention.

Overall, our evaluation focuses primarily on **Llama-3B**, with Qwen also included in the experimental evaluation. Gemma and Mistral are discussed here only to examine how the approach extends to other model families. Differences in hidden size and projection dimensions do not affect the basic masking approach, since the same mechanism is applied using the dimensions of each model’s linear projections. Model specific architectural differences mainly change the operations surrounding the outsourced projections rather than the masking principle itself. Therefore, the proposed mechanism can be applied in the same way to other models that contain linear projections suitable for outsourcing.

3.3 Differential Privacy

Differential privacy is a formal notion of privacy for databases introduced by Dwork et al. [12]. Since its inception many variants have been proposed, alongside an increasingly diverse range of applications. For this paper, we use a variant of differential privacy called f -differential privacy [10], as the noise addition mechanism in our scheme, called the Gaussian mechanism, has a simpler characterization under this definition than the usual definition of approximate differential privacy [12]. Furthermore, our definition is tailored to our use case, which considers privacy with respect to prompts with n tokens.

Let us begin with some notation. We denote vectors from $\mathbb{R}^d$, where d is a positive integer, as lowercase boldface letters, e.g., $\mathbf{x}$ and $\mathbf{y}$. The i th element of $\mathbf{x}$ is denoted by x_i . With this we may write $\mathbf{x}$ as

$$\mathbf{x} = (x_i)_{i \in [d]}$$

¹¹See <https://huggingface.co/collections/google/googles-gemma-models-family>.

¹²See <https://huggingface.co/mistralai>.

to emphasize its elements, where $[d] = \{1, 2, \dots, d\}$. For any $\mathbf{x} = (x_i)_{i \in [d]}$, we use the notation $|\mathbf{x}|$ to denote the vector $(|x_i|)_{i \in [d]}$. For any real number $p \geq 1$, the ℓ_p norm of $\mathbf{x} \in \mathbb{R}^d$ is defined as

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^d |x_i|^p \right)^{1/p}$$

We define a dataset X as an n -tuple each element of which is a vector from $\mathbb{R}^d$. Two datasets X_1 and X_2 are called *neighbouring datasets* if they differ in one token. The f -differential privacy framework is based on the hypothesis testing interpretation of differential privacy. Given the output of a mechanism $\mathcal{M}$, the goal is to distinguish between two competing hypotheses: the underlying data set being X_1 or X_2 . Let Q_1 and Q_2 denote the probability distributions of $\mathcal{M}(D_1)$ and $\mathcal{M}(D_2)$, respectively. Given any rejection rule $0 \leq \phi \leq 1$, the type-I and type-II errors are defined as follows [10]: $\alpha_\phi = \mathbb{E}_{Q_1}[\phi]$ and $\beta_\phi = 1 - \mathbb{E}_{Q_2}[\phi]$.

Definition 1 (Trade-off function [10]). For any two probability distributions Q_1 and Q_2 on the same space, the *trade-off function* $T(Q_1, Q_2) : [0, 1] \rightarrow [0, 1]$ is defined by

$$T(Q_1, Q_2)(\alpha) = \inf\{\beta_\phi : \alpha_\phi \leq \alpha\}$$

for all $\alpha \in [0, 1]$, where the infimum is taken over all (measurable) rejection rules.

A trade-off function gives the minimum achievable type-II error at any given level of type-I error. For a function to be a trade-off function, it must satisfy the following conditions.

Proposition 1 ([10]). A function $f : [0, 1] \rightarrow [0, 1]$ is a trade-off function if and only if f is convex, continuous and non-increasing, and $f(x) \leq 1 - x$ for all $x \in [0, 1]$.

Abusing notation, let $\mathcal{M}(X)$ denote the distribution of a mechanism $\mathcal{M}$ when given a dataset X as input.

Definition 2 (f -Differential Privacy (f -DP) [10]). Let f be a trade-off function. A mechanism $\mathcal{M}$ is said to be *f -label differentially private* if $T(\mathcal{M}(X_1), \mathcal{M}(X_2)) \geq f$ for all neighbouring data sets X_1 and X_2 .

For $\epsilon \geq 0$, let G_ϵ denote the trade-off function between two normal distributions $\mathcal{N}(0, 1)$ and $\mathcal{N}(0, \epsilon)$, i.e.,

$$G_\epsilon = T(\mathcal{N}(0, 1), \mathcal{N}(0, \epsilon))$$

Then

Definition 3 (ϵ -Gaussian Differential Privacy (ϵ -GDP) [10]). A mechanism $\mathcal{M}$ is said to satisfy ϵ -Gaussian Differential Privacy (ϵ -GDP) if it is G_ϵ -DP. That is,

$$T(\mathcal{M}(X_1), \mathcal{M}(X_2)) \geq G_\epsilon$$

for all neighbouring data sets X_1 and X_2 .

Definition 4 (Global ℓ_2 -Sensitivity). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be a function.¹³ Then its global ℓ_2 -sensitivity, denoted Δf , is defined as

$$\Delta f = \max_{\mathbf{x}, \mathbf{y} \in \mathbb{R}^d} \|f(\mathbf{x}) - f(\mathbf{y})\|_2$$

□

Our definition of global sensitivity may seem different from the usual definition of global sensitivity in literature which is defined over neighboring datasets (e.g., [13]). In our case, the dataset is a set of prompts of n tokens. We say that two prompts are neighboring prompts if they differ in one token. This implies that the d -dimensional embedding of the original token is replaced with that of another token. Therefore, we simply define global sensitivity as the maximum difference over all possible d -dimensional token embeddings. Thus, we can talk about applying the ϵ -GDP mechanism token-by-token, instead of defining it over the entire prompt of n tokens, as is defined next.

¹³From here onwards, we use the symbol f to denote different functions, as we no longer need to refer to f -DP

Proposition 2 (ϵ -GDP Mechanism [10]). *The mechanism $f(X) + \mathcal{N}(0, (\Delta f/\epsilon)^2 I_d)$ is ϵ -GDP where Δf is the global sensitivity of the function $f: \mathbb{R}^d \rightarrow \mathbb{R}^d$ and I_d is the $d \times d$ identity matrix.*

The notion of ϵ -GDP satisfies sequential composition.

Proposition 3 (Sequential Composition [10]). *The composition of n -fold sequential ϵ_i -GDP mechanisms is*

$$\sqrt{\epsilon_1^2 + \dots + \epsilon_n^2}\text{-GDP}$$

In particular, applying the ϵ -GDP mechanism of Proposition 2 on n tokens in the prompt makes the overall mechanism $\sqrt{n}\epsilon$ -GDP. Lastly, ϵ -GDP is also immune to post processing [10]. That is, applying a randomized map with an arbitrary range to the output of an ϵ -GDP mechanism is still ϵ -GDP.

4 Threat Model and Proposed System Architecture

In our computational model, a trained LLM is hosted on a remote server by a service provider which the user can query through a Web interface via prompts (see Figure 1). The user only receives the final output of the model, which is a sequence of tokens generated by the LLM. We assume that the server owned by the service provider has two execution environments: a trusted execution environment (TEE) and an untrusted execution environment powered by one or more GPUs, which we simply call GPU.

4.1 Privacy Goals and Threat model

We seek to provide two competing privacy goals

1. The user should not learn the parameters of the LLMs, which are considered the intellectual property (IP) of the server.
2. The service provider should not learn the contents of the user prompt.

The first goal is readily achieved in our computational model, as it is in prevalent deployments of LLMs, where the LLM is hosted remotely. Of course, the user may still be able to cleverly craft prompts to gather enough prompt-response pairs to deduce the model’s parameters through a model stealing attack [35]. However, this threat is beyond the scope of our work. We will therefore not mention this goal from here onwards, and instead focus on the second goal.

For the second goal we **assume** that the TEE is trusted in the sense that any information exchanged between the user and the TEE, and any computation done on the TEE cannot be eavesdropped, even by the service provider. This is a standard assumption for TEEs, as they are generally considered safe environments to perform tasks on sensitive data such as face recognition on smartphones.¹⁴ On the other hand, the GPU is considered honest-but-curious. Any information transmitted to the GPU, and any computation at the GPU will be carried out as specified, however this is visible to the service provider.

4.2 Adversarial Capabilities

Since the service provider hosts the LLM as a service, it knows all its parameters and architecture. Thus, we assume an adversary, an honest-but-curious service provider, who takes this information about its LLM and any information given to the GPU during LLM inference as input and attempts to reconstruct the user prompt. This is known as an embedding inversion attack (see, for example [27]). Note that in order to launch this attack, the adversary can build a dataset of known prompts and any information given to the GPU to train a model to successfully invert embeddings.

¹⁴See Apple’s Face ID technology as an example: <https://support.apple.com/en-au/102381>

4.3 Proposed System Architecture

Since we assume that the TEE is trusted, a straightforward way to achieve our main privacy goal is to do the entire LLM inference on the TEE without involving the GPU. However, there is a big performance mismatch between inference at the TEE versus the GPU. Most TEEs, such as the Intel TDX, do not provide GPU-based computation. As a result, the CPU-based computation at the TEE performs poorly compared to GPUs, since the computations involved in LLM inference are optimized for GPUs. We illustrate this in detail in Section 7, where we compare LLM inference times between the TEE (intel TDX) and GPU. For up to 800 generated tokens, the TEE takes between 53 to 83 seconds compared to the GPU which finishes the task within 3 to 5 seconds.

We are not the first to notice this disparity. Tramèr and Boneh [34] used this as the motivation to split inference for a conventional deep neural network (DNN) between a TEE and an untrusted external processor. The same observation was used by Xue et al [39] to motivate split-inference for LLMs. The main idea in [34] to split DNN inference, which was later adopted by Xue et al [39] for LLM inference, is to outsource linear operations to the GPU. The input to these linear operations can be *masked* using a fast cryptographic cipher. Due to the linearity of the operation, the result can be separated into the expected output from the original (unmasked) input and a randomized component, which can be undone upon receiving the result from the GPU. Moreover, part of the process can be carried out offline, and hence preprocessed.

4.3.1 Offloading Linear Operations

To make this more precise, let us first define a linear map. Let V and V' be vector spaces over a field K . A function $f : V \rightarrow V'$ is a linear map if for all $\mathbf{x}, \mathbf{y} \in V$ and scalars $a, b \in K$, f satisfies

$$f(a\mathbf{x} + b\mathbf{y}) = af(\mathbf{x}) + bf(\mathbf{y})$$

As an example related to our use case, let $\mathbf{x} \in \mathbb{R}^d$ be the input to any of the components of the LLM (Figure 2), e.g., the input embedding, where d is the embedding dimension. Let $\boldsymbol{\eta} \in \mathbb{R}^d$ be an arbitrary vector, which we call the *noise vector* or the *mask*. Define the vector $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\eta}$ as the *masked input*. Let W be a $d' \times d$ matrix of reals, where d' is potentially different from d . Then

$$W\tilde{\mathbf{x}} = W(\mathbf{x} + \boldsymbol{\eta}) = W\mathbf{x} + W\boldsymbol{\eta} = \mathbf{r}$$

holds because this is a linear map from the vector space of dimension d to the vector space of dimension d' over the reals. Thus, any operation in the LLM inference that involves a matrix multiplication is a linear operation. Note that the entries of the matrix are known both to the TEE and the GPU courtesy of the service provider, as these are learned weights of the LLM.

Now the TEE generates the noise vector, masks the input and hands over the masked input $\tilde{\mathbf{x}}$ to the GPU. The GPU can multiply it with the matrix W to obtain the quantity above, and return the result $\mathbf{r}$ back to the TEE. The TEE can then straightforwardly compute

$$\mathbf{r} - W\boldsymbol{\eta} = (W\mathbf{x} + W\boldsymbol{\eta}) - W\boldsymbol{\eta} = W\mathbf{x} \tag{1}$$

This *offloading* process speeds up overall computational time compared to computing the product $W\mathbf{x}$ at the TEE due to two main reasons. First, matrix multiplication is computationally expensive, and GPUs are highly optimized to perform them. Secondly, since W is known to the TEE as well, the noise vector $\boldsymbol{\eta}$ can be generated beforehand, and the quantity $W\boldsymbol{\eta}$ can be precomputed, meaning that all the TEE has to do is perform one vector addition to produce $\tilde{\mathbf{x}}$, and another vector subtraction to extract $W\mathbf{x}$ from the result returned by the GPU. Both of these operations are significantly faster than matrix multiplication at run-time. Figure 3 illustrates this.

Since this approach only relies on the linearity of the offloaded operation, it can be applied directly to matrix multiplications and even linear projections with a bias.¹⁵ On the other hand, non-linear operations such as softmax, SiLU, and element-wise multiplication are evaluated only on the TEE, because, in general, an additive mask cannot be removed after applying a non-linear function f , i.e.,

$$f(\mathbf{x} + \boldsymbol{\eta}) - f(\boldsymbol{\eta}) \neq f(\mathbf{x})$$

¹⁵A linear projection W with a bias produces $W\mathbf{x} + \mathbf{b}$ when given $\mathbf{x}$ as an input. Here $\mathbf{b}$ is the bias, which is technically called an affine map.

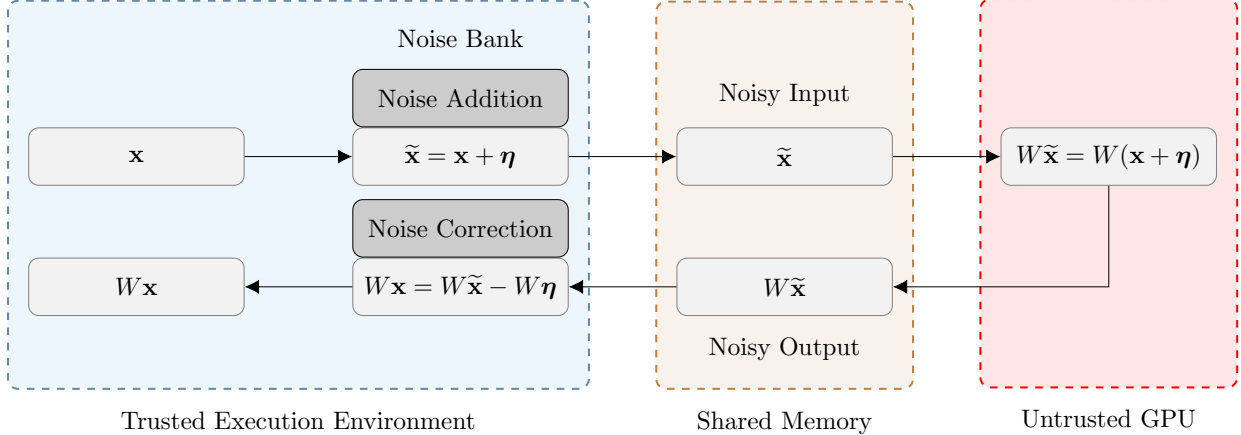


Figure 3: Proposed architecture of outsourcing expensive linear operations during LLM inference to the GPU and then removing the noise similar to [34]. In our scheme, the input to the GPU is protected via differentially private noise.

4.3.2 How to Mask the Inputs?

The approach taken by [34] and [39] is to mask the inputs through a stream cipher. To summarize, they encode (quantize) the input $\mathbf{x} \in \mathbb{R}^d$ as a vector of elements of a finite field of appropriate size, and then mask it by adding it to a vector each element of which is a uniformly random element of the finite field. This ensures that no information about the input is leaked to the entity receiving the masked input. Another reason to use operations over a finite field is to then be able to verify that the computations done at the untrusted entity are done correctly through Freivald’s verification protocol for matrix products [14]. However, quantizing the input and then subsequently doing the linear operations over the finite field introduces approximation errors due to quantization and rounding. This impacts the accuracy of the model. For instance, the accuracy of the quantized CNN models drops up to 0.5% in [34], whereas the quantized LLMs in [39] show an accuracy drop of up to 1.9%. In Section 7.3 we show that the approach from [34] applied to our problem introduces substantial changes in the LLM response.

Our idea is to instead mask the input using differential privacy. This has the advantage that masked operations can be performed in the native computational domain of LLMs, i.e., floating-point arithmetic, potentially maintaining accuracy.¹⁶ We further avoid the additional overhead of repeatedly converting the result between floating-point and finite-field representations if only the offloaded part of the computation is quantized. Differential privacy is a suitable solution in our case as we are not interested in verifying the computation done at the GPU which is assumed to be honest-but-curious. However, adding differentially private noise to the input is not straightforward, as we need to know the magnitude of the noise to be added. Recall that even if the GPU is honest-but-curious, it can still launch an embedding inversion attack, and if noise is not added to an appropriate scale, the adversary may be able to reconstruct the prompt. In the next section, we analyze the inputs to the linear operations we seek to outsource, in order to determine the amount of noise needed for privacy. In other words, we determine the global sensitivity (see Definition 4) of the operations preceding the linear components.

5 Differential Privacy Analysis

Figure 2 shows the components of LLM inference offloaded to the GPU, which are all linear operations. The sub-component “Down” is also a linear operation and hence a candidate for offloading. However, as we shall show in Section 6.5, the amount of noise needed to mask the input to this component is many orders of magnitude higher than the amount needed for inputs to other components, resulting in an overwhelming amplification of floating-point errors (see Section 5.5). Returning to the figure, the offloaded components

¹⁶Although we do run into floating-point error accumulation issues which we discuss in Section 5.5.

labeled (Q, K, V) , Gate/Up and Language Model all follow layer normalization. On the other hand, the O projection follows softmax attention, whereas the Down projection is preceded by SiLU activation. The purpose of this section is to upper bound the possible change in the input to each of these components over all possible input vectors. The idea is to check how much the values can change if any token is replaced by an arbitrary token, and hence a complete change in the embedding vector. Since these inputs are outputs of the preceding components, we find the global sensitivity of the functions computed in these preceding components. As discussed above, this reduces to finding the global sensitivity of layer normalization, the (softmax) attention mechanism and the (SiLU) activation function. We begin this section with necessary notation, prove some results that will be used in our analysis and then give a detailed global sensitivity analysis of these components. We then briefly discuss how the outsourced components can be made differentially private with the global sensitivity thus calculated. We finish the section with an important note on floating-point errors.

5.1 Preliminaries

The dot product of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, is defined as

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^d x_i y_i$$

and their entry-wise product is defined as

$$\mathbf{x} \odot \mathbf{y} = (x_i y_i)_{i \in [d]}$$

A vector which we will frequently encounter is the entry-wise product of a vector with itself denoted

$$\mathbf{x}^{\odot 2} = \mathbf{x} \odot \mathbf{x} = (x_i^2)_{i \in [d]} \quad (2)$$

We shall call this the *entry-wise square* of a vector. The Cauchy-Schwarz inequality states that

$$|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2$$

Let $\mathbf{x} \in \mathbb{R}^d$ and let W be a matrix of appropriate dimensions. Then

$$\|W\mathbf{x}\|_2 \leq \|W\|_2 \|\mathbf{x}\|_2 \quad (3)$$

where we define $\|W\|_2$ as the spectral norm of W , i.e., its largest singular value which can be obtained through its singular value decomposition. This inequality is a standard result; see, for example, [4].

5.2 Some Useful Results

In this section, we prove a few results and introduce a few notions that will be helpful in proving theorems in the ensuing section.

Proposition 4. *Let $\mathbf{x} \in \mathbb{R}^d$. Let*

$$\sigma_{\mathbf{x}} = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - c)^2}$$

where $c \in \mathbb{R}$. Define

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - c \cdot \mathbf{1}}{\sigma_{\mathbf{x}}}$$

where $\mathbf{1}$ is the d -element vector each element of which is 1. Then $\|\hat{\mathbf{x}}\|_2 = \sqrt{d}$.

Proof. We have

$$\begin{aligned}\|\mathbf{x}\|_2^2 &= \sum_{i=1}^d \left(\frac{x_i - c}{\sigma_{\mathbf{x}}} \right)^2 \\ &= \frac{1}{\sigma_{\mathbf{x}}^2} \sum_{i=1}^d (x_i - c)^2 \\ &= \frac{d}{\sum_{i=1}^d (x_i - c)^2} \sum_{i=1}^d (x_i - c)^2 = d\end{aligned}$$

□

Definition 5. Let $\mathbf{x} \in \mathbb{R}^d$, and let $\gamma > 0$ be a real number. The *z-score normalization* of $\mathbf{x}$ is

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu_{\mathbf{x}} \cdot \mathbf{1}}{\sqrt{\sigma_{\mathbf{x}}^2 + \gamma}}$$

where

$$\mu_{\mathbf{x}} = \frac{1}{d} \sum_{i=1}^d x_i \quad \sigma_{\mathbf{x}}^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu_{\mathbf{x}})^2$$

The *root mean square (RMS) normalization* of $\mathbf{x}$ is the z-score normalization of $\mathbf{x}$ with $\mu_{\mathbf{x}} = 0$. We say that $\hat{\mathbf{x}}$ is the normalized version of $\mathbf{x}$ if it is either z-score or RMS normalized. □

Proposition 5. Let $\mathbf{x} \in \mathbb{R}^d$ and let $\hat{\mathbf{x}}$ be its normalized version. Then $\|\hat{\mathbf{x}}\|_2 \leq \sqrt{d}$.

Proof. Since $\hat{\mathbf{x}}$ is normalized we have

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu_{\mathbf{x}} \cdot \mathbf{1}}{\sqrt{\sigma_{\mathbf{x}}^2 + \gamma}}$$

where $\mu_{\mathbf{x}} = \frac{1}{d} \sum_{i=1}^d x_i$ if it is z-score normalized and $\mu_{\mathbf{x}} = 0$ otherwise. We have

$$\|\hat{\mathbf{x}}\|_2 = \left(\sum_{i=1}^d \hat{x}_i^2 \right)^{\frac{1}{2}}$$

Consider the i th summand

$$\hat{x}_i^2 = \left(\frac{x_i - \mu_{\mathbf{x}}}{\sqrt{\sigma_{\mathbf{x}}^2 + \gamma}} \right)^2 = \frac{(x_i - \mu_{\mathbf{x}})^2}{\sigma_{\mathbf{x}}^2 + \gamma} \leq \frac{(x_i - \mu_{\mathbf{x}})^2}{\sigma_{\mathbf{x}}^2} = \left(\frac{x_i - \mu_{\mathbf{x}}}{\sigma_{\mathbf{x}}} \right)^2$$

Thus, from Proposition 4 either by setting $c = \mu_{\mathbf{x}} = \frac{1}{d} \sum_{i=1}^d x_i$ or by setting $c = \mu_{\mathbf{x}} = 0$ we have

$$\|\hat{\mathbf{x}}\|_2 \leq \left\| \frac{\mathbf{x} - \mu_{\mathbf{x}} \cdot \mathbf{1}}{\sigma_{\mathbf{x}}} \right\|_2 = \sqrt{d}.$$

□

Proposition 6. Let $\mathbf{x} \in \mathbb{R}^d$, and let $\mathbf{x}^{\odot 2}$ be its entry-wise square. Then

$$\|\mathbf{x}^{\odot 2}\|_2 \leq \|\mathbf{x}\|_2^2$$

Proof. We have

$$\mathbf{x} = (x_i)_{i \in [d]}$$

And from Eq. 2, by definition

$$\mathbf{x}^{\odot 2} = (x_i^2)_{i \in [d]}$$

Therefore

$$\|\mathbf{x}^{\odot 2}\|_2^2 = \sum_{i=1}^d (x_i^2)^2 = \sum_{i=1}^d x_i^4$$

Now,

$$\begin{aligned} (\|\mathbf{x}\|_2^2)^2 &= \left(\sum_{i=1}^d x_i^2 \right)^2 \\ &= \left(\sum_{i=1}^d x_i^2 \right) \left(\sum_{i=1}^d x_i^2 \right) \\ &= \sum_{i=1}^d \sum_{j=1}^d x_i^2 x_j^2 \\ &= \sum_{i=1}^d x_i^2 x_i^2 + \sum_{i=1}^d \sum_{j=1, j \neq i}^d x_i^2 x_j^2 \\ &= \sum_{i=1}^d x_i^4 + \sum_{i=1}^d \sum_{j=1, j \neq i}^d x_i^2 x_j^2 \\ &\geq \sum_{i=1}^d x_i^4 = \|\mathbf{x}^{\odot 2}\|_2^2, \end{aligned}$$

as the omitted term is the sum of non-negative numbers. □

Proposition 7. Let $m \geq 2$ be an integer. Let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m \in \mathbb{R}^d$, where

$$\mathbf{x}_i = (x_{ij})_{j \in [d]}$$

for $i \in [m]$. For $i \in [m]$, let $\mathbf{x}_i^{\odot 2}$ denote the entry-wise square of $\mathbf{x}_i$. Then

$$\|\mathbf{x}_1 \odot \mathbf{x}_2 \odot \dots \odot \mathbf{x}_m\|_2 \leq \sqrt{\|\mathbf{x}_1^{\odot 2}\|_2} \sqrt{\|\mathbf{x}_2^{\odot 2}\|_2} \dots \sqrt{\|\mathbf{x}_m^{\odot 2}\|_2}$$

Proof. We prove this by induction. Consider the base case with $m = 2$. Then note that

$$\|\mathbf{x}_1 \odot \mathbf{x}_2\|_2^2 = \sum_{j=1}^d x_{1j}^2 x_{2j}^2 = \langle \mathbf{x}_1^{\odot 2}, \mathbf{x}_2^{\odot 2} \rangle$$

Furthermore, $\langle \mathbf{x}_1^{\odot 2}, \mathbf{x}_2^{\odot 2} \rangle \geq 0$, as it is the dot product of two vectors all elements of which are non-negative. Thus from the Cauchy-Schwarz inequality we get

$$\langle \mathbf{x}_1^{\odot 2}, \mathbf{x}_2^{\odot 2} \rangle \leq \|\mathbf{x}_1^{\odot 2}\|_2 \cdot \|\mathbf{x}_2^{\odot 2}\|_2$$

This immediately gives us

$$\|\mathbf{x}_1 \odot \mathbf{x}_2\|_2 \leq \sqrt{\|\mathbf{x}_1^{\odot 2}\|_2} \sqrt{\|\mathbf{x}_2^{\odot 2}\|_2}$$

which proves the base case. Next assume that the statement is true for $m = k$, and consider

$$\|\mathbf{x}_1 \odot \mathbf{x}_2 \odot \dots \odot \mathbf{x}_{k+1}\|_2$$

Now $\mathbf{x}_1 \odot \mathbf{x}_2 \odot \dots \odot \mathbf{x}_k$ is a vector in $\mathbb{R}^d$. Denote this by $\mathbf{y}$. Then through the base case

$$\|\mathbf{x}_1 \odot \mathbf{x}_2 \odot \dots \odot \mathbf{x}_{k+1}\|_2 = \|\mathbf{y} \odot \mathbf{x}_{k+1}\|_2 \leq \sqrt{\|\mathbf{y}^{\odot 2}\|_2} \cdot \sqrt{\|\mathbf{x}_{k+1}^{\odot 2}\|_2}$$

From Proposition 6, we have

$$\sqrt{\|\mathbf{y}^{\odot 2}\|_2} \leq \sqrt{\|\mathbf{y}\|_2^2} = \|\mathbf{y}\|_2$$

Thus,

$$\begin{aligned} \|\mathbf{x}_1 \odot \mathbf{x}_2 \odot \cdots \odot \mathbf{x}_{k+1}\|_2 &\leq \|\mathbf{y}\|_2 \cdot \sqrt{\|\mathbf{x}_{k+1}^{\odot 2}\|_2} \\ &= \|\mathbf{x}_1 \odot \mathbf{x}_2 \odot \cdots \odot \mathbf{x}_k\|_2 \cdot \sqrt{\|\mathbf{x}_{k+1}^{\odot 2}\|_2} \\ &\leq \sqrt{\|\mathbf{x}_1^{\odot 2}\|_2} \sqrt{\|\mathbf{x}_2^{\odot 2}\|_2} \cdots \sqrt{\|\mathbf{x}_k^{\odot 2}\|_2} \sqrt{\|\mathbf{x}_{k+1}^{\odot 2}\|_2} \end{aligned}$$

which follows from the induction hypothesis. This proves the result. $\square$

Proposition 8. *Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ and let $\hat{\mathbf{y}}$ be the normalized version of $\mathbf{y}$. Then*

$$\|\mathbf{x} \odot \hat{\mathbf{y}}\|_2 \leq \sqrt{d} \cdot \|\mathbf{x}\|_4$$

Proof. From Proposition 7, we have

$$\|\mathbf{x} \odot \hat{\mathbf{y}}\|_2 \leq \sqrt{\|\mathbf{x}^{\odot 2}\|_2} \cdot \sqrt{\|\hat{\mathbf{y}}^{\odot 2}\|_2}$$

where $\mathbf{x}^{\odot 2}$ and $\hat{\mathbf{y}}^{\odot 2}$ are the entry-wise squares of $\mathbf{x}$ and $\hat{\mathbf{y}}$, respectively. From Proposition 6 we have

$$\|\hat{\mathbf{y}}^{\odot 2}\|_2 \leq \|\hat{\mathbf{y}}\|_2^2$$

And from Proposition 5 we have

$$\|\hat{\mathbf{y}}\|_2^2 \leq d$$

Thus,

$$\sqrt{\|\hat{\mathbf{y}}^{\odot 2}\|_2} \leq \sqrt{d}$$

Finally, note that

$$\begin{aligned} (\|\mathbf{x}^{\odot 2}\|_2)^{1/2} &= \left((x_1^2)^2 + (x_2^2)^2 + \cdots + (x_d^2)^2 \right)^{1/2} \\ &= (x_1^4 + x_2^4 + \cdots + x_d^4)^{1/4} \\ &= \|\mathbf{x}\|_4 \end{aligned}$$

Combining these we get

$$\|\mathbf{x} \odot \hat{\mathbf{y}}\|_2 \leq \sqrt{d} \cdot \|\mathbf{x}\|_4 \quad (4)$$

$\square$

5.3 Global Sensitivity Analysis

We now analyze global sensitivity (Definition 4) of the functions preceding the offloaded components. In our analysis, we use the same dimension d for all operations. As discussed in Section 3.2, some components use different dimensions, e.g., d_{key} . However, this is merely an implementation issue, and our sensitivity analysis can be used by replacing d with the exact dimension applicable to the operation.

5.3.1 Initial Tokens

As a warm-up we look at the initial layer, even though it is not offloaded. At initialization, the input prompt is converted into tokens, and then each token is represented by a token embedding $\mathbf{x} \in \mathbb{R}^d$. The token embeddings are from the vocabulary $\text{vocab} \subseteq \mathbb{R}^d$ of the LLM. Global sensitivity, in this case for an identity function, can be found experimentally as

$$\max_{\mathbf{x}, \mathbf{y} \in \text{vocab}} \|\mathbf{x} - \mathbf{y}\|_2$$

Note that we do not need to span the entire space $\mathbb{R}^d$, since the token embeddings are only from vocab .

5.3.2 After Layer Normalization

The layer normalization function $\text{LN} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ as defined in [5] is

$$\text{LN}(\mathbf{x}; \boldsymbol{\alpha}, \boldsymbol{\beta}, \gamma) = \boldsymbol{\alpha} \odot \hat{\mathbf{x}} + \boldsymbol{\beta} \quad (5)$$

where $\boldsymbol{\alpha}, \boldsymbol{\beta} \in \mathbb{R}^d$ are learned parameters, and $\hat{\mathbf{x}}$ is the z-score normalization of $\mathbf{x} \in \mathbb{R}^d$.¹⁷ In some models such as the new Llama models, $\hat{\mathbf{x}}$ is RMS normalized, known as the RMSNorm [40]. The benefit and use of the RMSNorm in Llama models is discussed in [17]. The following result holds for both standardization methods. We will mostly skip any reference to the parameters $\boldsymbol{\alpha}, \boldsymbol{\beta}$ and γ , and simply write $\text{LN}(\mathbf{x})$ instead of $\text{LN}(\mathbf{x}; \boldsymbol{\alpha}, \boldsymbol{\beta}, \gamma)$ to avoid notational clutter.

Theorem 1. *Let LN be as defined in Eq. (5). Then its global sensitivity ΔLN satisfies*

$$\Delta \text{LN} \leq 2\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4$$

Proof. For any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$,

$$\begin{aligned} \|\text{LN}(\mathbf{x}) - \text{LN}(\mathbf{y})\|_2 &= \|\boldsymbol{\alpha} \odot \hat{\mathbf{x}} + \boldsymbol{\beta} - \boldsymbol{\alpha} \odot \hat{\mathbf{y}} - \boldsymbol{\beta}\|_2 \\ &= \|\boldsymbol{\alpha} \odot (\hat{\mathbf{x}} - \hat{\mathbf{y}})\|_2 \\ &= \|\boldsymbol{\alpha} \odot \hat{\mathbf{x}} - \boldsymbol{\alpha} \odot \hat{\mathbf{y}}\|_2 \\ &\leq \|\boldsymbol{\alpha} \odot \hat{\mathbf{x}}\|_2 + \|\boldsymbol{\alpha} \odot \hat{\mathbf{y}}\|_2 \end{aligned}$$

where in the last step we have used the triangle inequality for the Euclidean norm. From Proposition 8 we get

$$\|\boldsymbol{\alpha} \odot \hat{\mathbf{x}}\|_2 \leq \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 \quad \text{and} \quad \|\boldsymbol{\alpha} \odot \hat{\mathbf{y}}\|_2 \leq \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4$$

Putting these inequalities in the above inequality, we get

$$\|\text{LN}(\mathbf{x}) - \text{LN}(\mathbf{y})\|_2 \leq \|\boldsymbol{\alpha} \odot \hat{\mathbf{x}}\|_2 + \|\boldsymbol{\alpha} \odot \hat{\mathbf{y}}\|_2 \leq 2\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4$$

Since $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ were arbitrary, we conclude

$$\Delta \text{LN} \leq 2\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4$$

□

5.3.3 After the Attention Mechanism

The attention mechanism can be defined as follows. Let W_{qry} , W_{key} and W_{val} be $d \times d$ matrices. Suppose we have n vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$. For $1 \leq i \leq n$, define

$$\mathbf{q}_i = W_{\text{qry}} \cdot \mathbf{x}_i, \quad \mathbf{k}_i = W_{\text{key}} \cdot \mathbf{x}_i, \quad \mathbf{v}_i = W_{\text{val}} \cdot \mathbf{x}_i$$

which are again vectors from $\mathbb{R}^d$. Let $\mathbf{q} = W_{\text{qry}} \mathbf{x}$ for some $\mathbf{x} \in \mathbb{R}^d$. For $1 \leq i \leq n$ define

$$p_i(\mathbf{x}) = \frac{\exp(\langle \mathbf{q}, \mathbf{k}_i \rangle / \sqrt{d})}{\sum_{j=1}^n \exp(\langle \mathbf{q}, \mathbf{k}_j \rangle / \sqrt{d})} = \text{softmax} \left(\frac{\langle \mathbf{q}, \mathbf{k}_i \rangle}{\sqrt{d}} \right)$$

Note that $\sum_{i=1}^n p_i(\mathbf{x}) = 1$. With this, the attention mechanism for any $\mathbf{x} \in \mathbb{R}^d$ is defined as

$$\text{att}(\mathbf{x}) = \sum_{i=1}^n p_i(\mathbf{x}) \mathbf{v}_i \quad (6)$$

¹⁷The parameter γ in the z-score normalization as defined in Definition 5 does not appear in [5] but it is used in actual implementations for numerical stability to ensure the denominator never becomes 0.

The input to the attention function is in fact layer normalized. Thus, we can write it as

$$\begin{aligned}
\text{att}(\mathbf{x}) &= \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot \text{LN}(\mathbf{x}_i) \\
&= \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i + \boldsymbol{\beta}) \\
&= \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i) + \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot \boldsymbol{\beta} \\
&= \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i) + \left(\sum_{i=1}^n p_i(\widehat{\mathbf{x}}) \right) W_{\text{val}} \cdot \boldsymbol{\beta} \\
&= W_{\text{val}} \cdot \boldsymbol{\beta} + \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i)
\end{aligned} \tag{7}$$

where $\widehat{\mathbf{x}}$ and $\widehat{\mathbf{x}}_i$ are normalized versions of $\mathbf{x}$ and $\mathbf{x}_i$ respectively. We are interested in finding Δatt .

Theorem 2. *Let att be as defined in Eq. (7). Then its global sensitivity Δatt satisfies*

$$\Delta \text{att} \leq 2\sqrt{d} \cdot \|W_{\text{val}}\|_2 \cdot \|\boldsymbol{\alpha}\|_4$$

Proof. By definition, for any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$

$$\text{att}(\mathbf{x}) = W_{\text{val}} \cdot \boldsymbol{\beta} + \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i)$$

and

$$\text{att}(\mathbf{y}) = W_{\text{val}} \cdot \boldsymbol{\beta} + \sum_{i=1}^n p'_i(\widehat{\mathbf{y}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}'_i)$$

Note that we use different variables p'_i and $\widehat{\mathbf{x}}'_i$ for $\text{att}(\mathbf{y})$ since replacing $\mathbf{x}$ with $\mathbf{y}$ changes the softmax probabilities $p_i(\widehat{\mathbf{x}})$ and the sequence of vectors $\widehat{\mathbf{x}}_i$ due to one vector being different. Now

$$\begin{aligned}
\|\text{att}(\mathbf{x}) - \text{att}(\mathbf{y})\|_2 &= \left\| \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i) - \sum_{i=1}^n p'_i(\widehat{\mathbf{y}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}'_i) \right\|_2 \\
&\leq \left\| \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i) \right\|_2 + \left\| \sum_{i=1}^n p'_i(\widehat{\mathbf{y}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}'_i) \right\|_2 \\
&\leq \sum_{i=1}^n \|p_i(\widehat{\mathbf{x}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i)\|_2 + \sum_{i=1}^n \|p'_i(\widehat{\mathbf{y}}) W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}'_i)\|_2 \\
&= \sum_{i=1}^n p_i(\widehat{\mathbf{x}}) \|W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i)\|_2 + \sum_{i=1}^n p'_i(\widehat{\mathbf{y}}) \|W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}'_i)\|_2
\end{aligned} \tag{8}$$

where we have used standard properties of the Euclidean norm. Now let us examine first of the two norms above in isolation. Let $1 \leq i \leq n$ be fixed. From Eq. (3), We have

$$\begin{aligned}
\|W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i)\|_2 &\leq \|W_{\text{val}}\|_2 \cdot \|\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}_i\|_2 \\
&\leq \|W_{\text{val}}\|_2 \cdot \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4
\end{aligned}$$

where the second inequality follows from Proposition 5. Likewise, for any $i \in \{1, \dots, n\}$, we have

$$\|W_{\text{val}} \cdot (\boldsymbol{\alpha} \odot \widehat{\mathbf{x}}'_i)\|_2 \leq \|W_{\text{val}}\|_2 \cdot \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4$$

Thus, Eq. (8) becomes

$$\begin{aligned}
\|\text{att}(\mathbf{x}) - \text{att}(\mathbf{y})\|_2 &\leq \sum_{i=1}^n p_i(\hat{\mathbf{x}}) \|W_{\text{val}}\|_2 \cdot \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \sum_{i=1}^n p'_i(\hat{\mathbf{y}}) \|W_{\text{val}}\|_2 \cdot \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 \\
&= \left(\sum_{i=1}^n p_i(\hat{\mathbf{x}}) \right) \|W_{\text{val}}\|_2 \cdot \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \left(\sum_{i=1}^n p'_i(\hat{\mathbf{y}}) \right) \|W_{\text{val}}\|_2 \cdot \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 \\
&= 2\sqrt{d} \cdot \|W_{\text{val}}\|_2 \cdot \|\boldsymbol{\alpha}\|_4
\end{aligned}$$

Since $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ were arbitrary, we conclude

$$\Delta \text{att} \leq 2\sqrt{d} \cdot \|W_{\text{val}}\|_2 \cdot \|\boldsymbol{\alpha}\|_4$$

□

5.3.4 After the Activation Function

The activation function in the feed-forward neural network can be defined as follows. Let W_{gate} and W_{up} be two $d \times d$ matrices. Let $g(\mathbf{x}) = W_{\text{gate}} \cdot \mathbf{x}$ for $\mathbf{x} \in \mathbb{R}^d$. For any $x \in \mathbb{R}$ the *sigmoid* function is defined as

$$\text{sig}(x) = \frac{1}{1 + e^{-x}} \quad (9)$$

Note that for any $x \in \mathbb{R}$, $\text{sig}(x) \in (0, 1)$. We extend this to a vector $\mathbf{x} \in \mathbb{R}^d$ by defining $\text{sig}(\mathbf{x})$ to be the d -dimensional vector whose i th element is $\text{sig}(x_i)$. With this the activation function on $\mathbf{x}$ is defined as

$$\text{act}(\mathbf{x}) = W_{\text{up}} \cdot \mathbf{x} \odot W_{\text{gate}} \cdot \mathbf{x} \odot \text{sig}(g(\mathbf{x}))$$

The term $W_{\text{gate}} \cdot \mathbf{x} \odot \text{sig}(g(\mathbf{x}))$ is known as the sigmoid linear unit or SiLU. Just like the attention mechanism, this function is applied on the normalized version of $\mathbf{x}$. Therefore, we can write the activation function as

$$\text{act}(\mathbf{x}) = W_{\text{up}} \cdot \text{LN}(\mathbf{x}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{x}) \odot \text{sig}(g(\mathbf{x})) \quad (10)$$

where we have re-defined $g(\mathbf{x})$ as $W_{\text{gate}} \cdot \text{LN}(\mathbf{x})$. With this, we have the following result for Δact .

Theorem 3. *Let act be as defined in Eq. (10). Then its global sensitivity Δact satisfies*

$$\Delta \text{act} \leq 2\sqrt{d} \cdot \left(\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2 \right)^2 \cdot \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2$$

Proof. For any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, we have

$$\begin{aligned}
\|\text{act}(\mathbf{x}) - \text{act}(\mathbf{y})\|_2 &= \|W_{\text{up}} \cdot \text{LN}(\mathbf{x}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{x}) \odot \text{sig}(g(\mathbf{x})) \\
&\quad - W_{\text{up}} \cdot \text{LN}(\mathbf{y}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{y}) \odot \text{sig}(g(\mathbf{y}))\|_2 \\
&\leq \|W_{\text{up}} \cdot \text{LN}(\mathbf{x}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{x}) \odot \text{sig}(g(\mathbf{x}))\|_2 \\
&\quad + \|W_{\text{up}} \cdot \text{LN}(\mathbf{y}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{y}) \odot \text{sig}(g(\mathbf{y}))\|_2
\end{aligned} \quad (11)$$

which follows from the triangle inequality of the Euclidean norm. Consider the first of the two norms above

$$\begin{aligned}
&\|W_{\text{up}} \cdot \text{LN}(\mathbf{x}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{x}) \odot \text{sig}(g(\mathbf{x}))\|_2 \\
&\leq \sqrt{\|(W_{\text{up}} \cdot \text{LN}(\mathbf{x}))^{\odot 2}\|_2} \cdot \sqrt{\|(W_{\text{gate}} \cdot \text{LN}(\mathbf{x}))^{\odot 2}\|_2} \cdot \sqrt{\|(\text{sig}(g(\mathbf{x})))^{\odot 2}\|_2} \\
&\leq \|W_{\text{up}} \cdot \text{LN}(\mathbf{x})\|_2 \cdot \|W_{\text{gate}} \cdot \text{LN}(\mathbf{x})\|_2 \cdot \|\text{sig}(g(\mathbf{x}))\|_2 \\
&\leq \|W_{\text{up}}\|_2 \cdot \|\text{LN}(\mathbf{x})\|_2 \cdot \|W_{\text{gate}}\|_2 \cdot \|\text{LN}(\mathbf{x})\|_2 \cdot \|\text{sig}(g(\mathbf{x}))\|_2 \\
&= \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2 \cdot \|\text{LN}(\mathbf{x})\|_2^2 \cdot \|\text{sig}(g(\mathbf{x}))\|_2
\end{aligned} \quad (12)$$

The first inequality follows from Proposition 7, the second from Proposition 6, and the third from Eq. (3). Now from Eq. (5),

$$\begin{aligned}\|\text{LN}(\mathbf{x})\|_2 &= \|\boldsymbol{\alpha} \odot \hat{\mathbf{x}} + \boldsymbol{\beta}\|_2 \\ &\leq \|\boldsymbol{\alpha} \odot \hat{\mathbf{x}}\|_2 + \|\boldsymbol{\beta}\|_2 \\ &\leq \sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2\end{aligned}$$

where the last inequality follows from Proposition 8. Thus,

$$\|\text{LN}(\mathbf{x})\|_2^2 \leq \left(\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2\right)^2 \quad (13)$$

For the other term, we have

$$\begin{aligned}\|\text{sig}(g(\mathbf{x}))\|_2^2 &= \sum_{i=1}^d (\text{sig}((g(\mathbf{x}))_i))^2 \\ &\leq \sum_{i=1}^d 1^2 \\ &= d\end{aligned} \quad (14)$$

Therefore $\|\text{sig}(g(\mathbf{x}))\|_2 \leq \sqrt{d}$. Thus, Eq. (12) becomes

$$\begin{aligned}\|W_{\text{up}} \cdot \text{LN}(\mathbf{x}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{x}) \odot \text{sig}(g(\mathbf{x}))\|_2 &\leq \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2 \cdot \|\text{LN}(\mathbf{x})\|_2^2 \cdot \|\text{sig}(g(\mathbf{x}))\|_2 \\ &\leq \sqrt{d} \cdot \left(\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2\right)^2 \cdot \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2\end{aligned}$$

Exactly the same way, we have

$$\|W_{\text{up}} \cdot \text{LN}(\mathbf{y}) \odot W_{\text{gate}} \cdot \text{LN}(\mathbf{y}) \odot \text{sig}(g(\mathbf{y}))\|_2 \leq \sqrt{d} \cdot \left(\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2\right)^2 \cdot \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2$$

Therefore, Eq. (11) becomes

$$\|\text{act}(\mathbf{x}) - \text{act}(\mathbf{y})\|_2 \leq 2\sqrt{d} \cdot \left(\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2\right)^2 \cdot \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2$$

Since $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ were arbitrary, we conclude

$$\Delta \text{act} \leq 2\sqrt{d} \cdot \left(\sqrt{d} \cdot \|\boldsymbol{\alpha}\|_4 + \|\boldsymbol{\beta}\|_2\right)^2 \cdot \|W_{\text{up}}\|_2 \cdot \|W_{\text{gate}}\|_2$$

□

5.4 Making the Process Differentially Private

Given the ℓ_2 -sensitivity Δf of the output $\mathbf{x}$ of the component f , which serves as the input to the component being offloaded to the GPU, we first sample noise $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, (\Delta f/\epsilon)^2 I_d)$, where I_d is the $d \times d$ identity matrix. We then offload the vector $\mathbf{x} + \boldsymbol{\eta}$ to the GPU. For a prompt of n tokens, with a *single* offloaded operation, this mechanism satisfies $\sqrt{n}\epsilon$ -Gaussian differential privacy according to Propositions 2 and 3. The steps taken by the GPU maintain this guarantee due to the post-processing property of GDP. However, note that a given prompt is offloaded multiple times: once for each offloaded component, once for each transformer layer, and then once for each generated token. Thus, to be precise, the overall privacy guarantee should include all offloaded instances. However, we stick to the privacy guarantee of $\sqrt{n}\epsilon$ -GDP and even ϵ -GDP (i.e., keeping the privacy parameter constant, instead of scaling it with the prompt size), as otherwise an overwhelming amount of noise needs to be added, and moreover, our prompt reconstruction attacks (Section 7.4) show negligible success rate with this privacy level.

5.5 Floating Point Issues

The noise addition and cancellation procedure in our scheme as shown in Eq. (1) works perfectly for real numbers. However, in floating-point arithmetic, this may result in a phenomenon known as *catastrophic cancellation* (see for example [28, §4.2]). Given the quantities $W\mathbf{x}$ and $W(\mathbf{x} + \boldsymbol{\eta}) - W\boldsymbol{\eta}$ we are interested in knowing the difference between the two, when the arithmetic is done over floating point numbers. The i th element of $W\mathbf{x}$ is given as $\langle \mathbf{w}_i, \mathbf{x} \rangle$, where $\mathbf{w}_i$ is the i th row of W . We thus consider a generic element of $W\mathbf{x}$, denoted $\langle \mathbf{w}, \mathbf{x} \rangle$, as the dot product of the appropriate row of W . The corresponding value through offloading is $\langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle - \langle \mathbf{w}, \boldsymbol{\eta} \rangle$.

To make this precise, we will use the notation fl to denote the variant of any function that uses two floating point numbers and outputs a floating point number as an answer. For example if x and y are floating point numbers, then $\text{fl}(x + y)$ denotes the operation that adds two floating point numbers and rounds them to the nearest floating point number. Under this notation, we are interested in the quantity

$$|\text{fl}(\langle \mathbf{w}, \mathbf{x} \rangle) - \text{fl}(\text{fl}(\langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle) - \text{fl}(\langle \mathbf{w}, \boldsymbol{\eta} \rangle))| \quad (15)$$

Note that the outer subtraction is not over floating point numbers.

5.5.1 Background On Floating Point Arithmetic

We give a brief background on floating point arithmetic summarized from [28]. We use the binary32 number format as the foundation. A number x in this format is written as

$$x = M \cdot 2^{e-p+1},$$

where $p = 24$ is the precision, e is an integer satisfying $e_{\min} \leq e \leq e_{\max}$ and M is an integer satisfying $2^{p-1} \leq |M| < 2^p$. To be precise this is called the normalized representation. For binary32 numbers we have $e_{\min} = -126$ and $e_{\max} = 127$. The *unit round-off* of radix-2 precision- p floating point system in the round-to-nearest mode is defined as

$$u = \frac{1}{2}2^{1-p} = 2^{-p} \quad (16)$$

This gives rise to the following standard model of floating-point arithmetic. For all floating-point numbers a, b such that $a * b$ does not underflow nor overflow, we have

$$\text{fl}(a * b) = (a * b)(1 + \delta), \quad |\delta| < u \quad (17)$$

where $*$ is one of the elementary arithmetic operations: $+, -, \times, \div$. Roughly, an underflow (respectively, overflow) occurs when the result is smaller (respectively, larger) than the smallest (respectively, largest) possible floating point number in the given number format.

The following definition is from [18, §3.1] (see also [28, §5.1])

Definition 6. Let δ_i satisfy $|\delta_i| \leq u$ for $1 \leq i \leq d$, and assume that

$$du < 1.$$

Then

$$\prod_{i=1}^d (1 + \delta_i)^{\pm 1} = 1 + \theta_d,$$

where

$$|\theta_d| \leq \frac{du}{1 - du} =: \gamma_d.$$

□

We will make use of the following result from [18, §3.1]:

$$|\langle \mathbf{w}, \mathbf{x} \rangle - \text{fl}(\langle \mathbf{w}, \mathbf{x} \rangle)| \leq \gamma_d \langle |\mathbf{w}|, |\mathbf{x}| \rangle \quad (18)$$

5.5.2 Floating-Point Error Bound

Our main result is as follows

Theorem 4. *If no underflow or overflow occurs then*

$$|\text{fl}(\langle \mathbf{w}, \mathbf{x} \rangle) - \text{fl}(\text{fl}(\langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle) - \text{fl}(\langle \mathbf{w}, \boldsymbol{\eta} \rangle))| \leq \gamma_d \langle |\mathbf{w}|, |\mathbf{x}| + |\mathbf{x} + \boldsymbol{\eta}| + |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle + O(du^2)$$

Proof. Let us define,

$$A = \text{fl}(\langle \mathbf{w}, \mathbf{x} \rangle), B = \text{fl}(\langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle), C = \text{fl}(\langle \mathbf{w}, \boldsymbol{\eta} \rangle)$$

With this notation, we seek to find the bound on

$$|A - \text{fl}(B - C)|$$

From one application of the standard model (Eq. (17)) we have

$$\begin{aligned} |A - \text{fl}(B - C)| &\leq |A - (B - C)(1 + \delta)| \\ &= |A - \langle \mathbf{w}, \mathbf{x} \rangle + \langle \mathbf{w}, \mathbf{x} \rangle - (B - C)(1 + \delta)| \\ &\leq |A - \langle \mathbf{w}, \mathbf{x} \rangle| + |(B - C)(1 + \delta) - \langle \mathbf{w}, \mathbf{x} \rangle| \end{aligned} \quad (19)$$

where the last step follows from the triangle inequality. From Eq. (18), we have

$$|A - \langle \mathbf{w}, \mathbf{x} \rangle| = |\text{fl}(\langle \mathbf{w}, \mathbf{x} \rangle) - \langle \mathbf{w}, \mathbf{x} \rangle| \leq \gamma_d \langle |\mathbf{w}|, |\mathbf{x}| \rangle \quad (20)$$

We expand the second term in Eq. (19) as follows

$$\begin{aligned} |(B - C)(1 + \delta) - \langle \mathbf{w}, \mathbf{x} \rangle| &= |B - C + \delta(B - C) - \langle \mathbf{w}, \mathbf{x} \rangle| \\ &\leq |B - C - \langle \mathbf{w}, \mathbf{x} \rangle| + |\delta(B - C)| \\ &= |B - C - \langle \mathbf{w}, \mathbf{x} \rangle + \langle \mathbf{w}, \boldsymbol{\eta} \rangle - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| + |\delta(B - C)| \\ &\leq |B - \langle \mathbf{w}, \mathbf{x} \rangle - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| + |C - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| + |\delta(B - C)| \\ &= |B - \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle| + |C - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| + |\delta(B - C)| \end{aligned} \quad (21)$$

with multiple applications of the triangle inequality. From Eq. 17, we have

$$|B - \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle| = |\text{fl}(\langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle) - \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle| \leq \gamma_d \langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| \rangle \quad (22)$$

and

$$|C - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| = |\text{fl}(\langle \mathbf{w}, \boldsymbol{\eta} \rangle) - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| \leq \gamma_d \langle |\mathbf{w}|, |\boldsymbol{\eta}| \rangle \quad (23)$$

The remaining term in Eq. (21) can be expanded as follows, noting that $|\delta| \leq u$ from Definition 6,

$$\begin{aligned} |\delta(B - C)| &\leq |\delta| |B - C| \\ &\leq u |B - C| \\ &= u |B - C + \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle - \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle| \\ &\leq u |B - \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle| + u |C - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| + u |\langle \mathbf{w}, \mathbf{x} \rangle| \\ &\leq u |B - \langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle| + u |C - \langle \mathbf{w}, \boldsymbol{\eta} \rangle| + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle \end{aligned}$$

which again follows from repeated use of the triangle inequality. From Eqs.(22) and (23), we have

$$\begin{aligned} |\delta(B - C)| &\leq u \gamma_d \langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| \rangle + u \gamma_d \langle |\mathbf{w}|, |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle \\ &= u \gamma_d \langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| + |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle \end{aligned} \quad (24)$$

By using the fact that

$$\frac{1}{1 - x} = 1 + x + x^2 + \dots$$

and denoting $\langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| + |\boldsymbol{\eta}| \rangle = c$ as a constant, we see that the first term in Eq. (24) becomes

$$\begin{aligned}
u\gamma_d \langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| + |\boldsymbol{\eta}| \rangle &= cu\gamma_d \\
&= cu \frac{du}{1 - du} \\
&= cdu^2 \frac{1}{1 - du} \\
&= cdu^2(1 + du + d^2u^2 + \dots) \\
&= cdu^2 + cd^2u^3 + cd^3u^4 + \dots \\
&= \mathcal{O}(du^2)
\end{aligned}$$

where we have used the definition of γ_d from Definition 6. Thus, Eq. (24) becomes

$$|\delta(B - C)| \leq u \langle |\mathbf{w}|, |\mathbf{x}| \rangle + \mathcal{O}(du^2) \quad (25)$$

Putting the results from Eqs. (22), (23) and (25) in Eq (21), we get

$$|(B - C)(1 + \delta) - \langle \mathbf{w}, \mathbf{x} \rangle| \leq \gamma_d \langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| \rangle + \gamma_d \langle |\mathbf{w}|, |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle + \mathcal{O}(du^2) \quad (26)$$

Finally, putting the results from Eqs. (20) and (26) in Eq. 19, we get

$$\begin{aligned}
|A - \text{fl}(B - C)| &\leq \gamma_d \langle |\mathbf{w}|, |\mathbf{x}| \rangle + \gamma_d \langle |\mathbf{w}|, |\mathbf{x} + \boldsymbol{\eta}| \rangle + \gamma_d \langle |\mathbf{w}|, |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle + \mathcal{O}(du^2) \\
&= \gamma_d \langle |\mathbf{w}|, |\mathbf{x}| + |\mathbf{x} + \boldsymbol{\eta}| + |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle + \mathcal{O}(du^2)
\end{aligned}$$

as desired. $\square$

How may we use the result of Theorem 4? Since $\mathbf{w}$, d and u are constants, the error bound depends on $\mathbf{x}$ and $\boldsymbol{\eta}$. The latter is a Gaussian random variable with mean $\mathbf{0}$ and scale $\frac{\Delta f}{\epsilon} I_d$. Furthermore, the range of values of $\mathbf{x}$ are known, e.g., after layer normalization. Thus, through a Monte Carlo simulation we can determine the error bound (ignoring the $\mathcal{O}(du^2)$ term) for a given value of ϵ . For a given value of ϵ , we can determine whether the overall error is acceptable in the sense that it will not change the recovered dot product by too much. Through our simulations, we found the following bound to be closer to the true error

$$|\text{fl}(\langle \mathbf{w}, \mathbf{x} \rangle) - \text{fl}(\text{fl}(\langle \mathbf{w}, \mathbf{x} + \boldsymbol{\eta} \rangle) - \text{fl}(\langle \mathbf{w}, \boldsymbol{\eta} \rangle))| \leq \gamma_{\lceil \log_2 d \rceil + 1} \langle |\mathbf{w}|, |\mathbf{x}| + |\mathbf{x} + \boldsymbol{\eta}| + |\boldsymbol{\eta}| \rangle + u \langle |\mathbf{w}|, |\mathbf{x}| \rangle \quad (27)$$

which is based on Higham’s estimate [18, §3.1, p. 64] for computing the dot product through pairwise summation. We will therefore use this bound in our experiments. Note that the only quantity in the bound above that depends on the specific floating-point number format is u which, as defined in Eq. (16), depends on the precision p of the format. Thus, this bound is transferrable to any floating-point number format such as binary32, binary16 (half-precision floating point format) or brain floating point (bfloat16).¹⁸

6 Implementation

We use Intel Trust Domain Extensions (TDX) to provide our TEE, a CPU-based trusted execution environment technology [6]. The Intel TDX architecture allows the creation of hardware-isolated virtual machines, which we call TDX guests,¹⁹ whose memory and execution state are protected from the hypervisor, other TDX guests, and any other software running on the host. We implement the split-inference architecture on a single physical machine by partitioning LLM inference between a TDX guest (virtual machine) and an semi-trusted GPU running on the host. The system is equipped with an Intel Xeon Silver 4514Y processor with 16 physical cores and approximately 117 GB of system memory. The TDX-protected virtual machine

¹⁸See details of these number formats and their use in LLM training and inference in <https://sebastianraschka.com/blog/2023/llm-mixed-precision-copy.html>.

¹⁹They are also called Trust Domains (TDs).

is configured with 32 virtual CPUs and approximately 93 GB of memory, while the remaining system memory is available to the host. The host is equipped with an NVIDIA RTX PRO 4500 Blackwell GPU with approximately 32 GB of device memory. Both environments run Ubuntu 24.04.

The TDX retains all values that must remain confidential, including intermediate activations, masking tensors, and correction values. The TDX also controls the autoregressive token generation process, also known as decoding, and manages the key-value cache (KV cache), which avoids recomputing attention over previous tokens at each decoding step. The partitioning only changes the physical location at which selected matrix multiplications are evaluated. It does not change the transformer architecture, model parameters, tensor dimensions,²⁰ or ordering of operations. After each offloaded operation is completed, the TDX guest reconstructs the same linear output that would have been produced by local execution and continues the original transformer computation.

Intel TDX provides two types of memory for the protected virtual machine. Private memory is encrypted and integrity protected and can only be accessed from within the protected TDX environment. Shared memory is used to communicate with entities outside the TDX guest, and does not receive the same protection [6]. We describe the usage of the shared memory in our system in Section 6.3.

6.1 Offloaded Operations

The components of the LLM offloaded to the GPU are illustrated in Figure 2. All these components involve matrix multiplications, specifically, with the matrices: $W_{\text{qry}}, W_{\text{key}}, W_{\text{val}}, W_{\text{oh}}, W_{\text{gate}}, W_{\text{up}},$ and W_{lm} . See Section 3.2 for a description of these matrices. We shall denote a generic matrix belonging to this set by W . The GPU simply multiplies these matrices with the corresponding masked inputs and returns the result back to TEE. The masked input and the result are shared between the TEE and the GPU through a shared memory. This is illustrated in Figure 3 for a generic matrix W .

6.2 Precomputing the Mask and Noise Correction

Instead of generating the noise η and the product $W\eta$ at inference time, we generate them in advance. This is essentially a time-memory tradeoff, and it is the same strategy used in [34] and [39]. In particular, $W\eta$ is itself a matrix product, and computing it at runtime would defeat the purpose of offloading expensive operations to the GPU. Note that we cannot offload this multiplication to the GPU as it reveals the noise vector, violating the differential privacy guarantee. For any offloaded component, equivalently, for any matrix W associated with the offloaded component, we call η the *mask* and $W\eta$ the *correction*, as this is the quantity the TEE has to subtract from the result obtained from the GPU. We generate the masks and corrections in advance for each offloaded component and store them in a *noise bank*. We maintain separate correction banks for (Q, K, V) projections, the O projection, the Gate/Up projections, and the language head projection.

The (Q, K, V) projections receive the same input (see Figure 2). A single mask can therefore be applied to their shared input, with three corresponding corrections stored for the three weight matrices. Similarly, the Gate and Up projections share the same input and therefore need a single mask, with two corrections corresponding to the two matrices. The O projection and language model head use one mask and one correction each. The noise bank is *indexed* by the input shape. As mentioned in Section 3.2, the matrices have different dimensions, resulting in outputs of different dimensions (also called tensor dimensions). Thus, during inference, the TEE retrieves an entry matching the current tensor dimensions, which serves as its index. Each noise entry, the mask and its corresponding correction, is consumed only once, preventing reuse of the same mask across different calls to these components. All masks and corrections are precomputed inside the TDX protected virtual machine and written to its local disk. Before inference, the entries required for the current batch are loaded from disk into protected memory. During inference, these loaded entries are used one at a time and discarded after use.

²⁰The term tensor refers to the multidimensional array used to represent data within an LLM during inference.

6.3 Shared Memory Communication Between TDX Guest and GPU Worker

Our TDX guest runs under QEMU (Quick Emulator), which provides Inter-VM Shared Memory (IVSHMEM)²¹ as a mechanism for exposing a shared-memory region between the host and the guest. The masked input to the outsourced linear operation, and the corresponding output are exchanged through this region between the TDX guest and the GPU, as shown in Figure 3. The host maps the shared-memory object into its address space, while the TDX guest maps the corresponding IVSHMEM device memory region into its own address space. Both sides can therefore access the same memory area, allowing large data transfers without serializing or transmitting them through the socket connection. Note that the shared-memory region contains only the masked input, and the corresponding output on the masked input from the GPU. The unmasked input, the mask, and the corresponding correction, are not written to the shared-memory region and remain inside the TDX guest. All masks and corrections are precomputed inside the protected virtual machine of the TDX guest, and written to its local disk. During inference, the required entries are loaded from this storage into the TDX guest’s protected memory, i.e., private memory.

Because the shared memory region is used only for exchanging the masked input and output, a separate mechanism is required to coordinate each outsourced computation. For this purpose, we use a TCP socket as a control channel between the TDX guest and the host. Control messages specify the metadata needed to locate and interpret the masked tensor in shared memory. The projection name identifies the requested operation, while the transformer-layer index identifies the correct weight matrix, since each layer has its own distinct set of weights. The shared-memory offset specifies where the masked tensor begins. Because the shared memory region holds only raw bytes with no inherent structure, its shape and datatype are also included, so the GPU worker can correctly interpret the stored values as a tensor. The byte length is included to verify that the expected amount of data is being accessed. Therefore, the masked input and corresponding output are transferred through IVSHMEM, while the TCP socket is used only to coordinate the outsourced computation.

6.4 Inference Models

We evaluate our approach using two instruction-tuned LLMs: **Llama-3B** and **Qwen3-4B-Instruct**, which we simply call **Qwen-4B** from now onwards.²² Both are decoder-only transformer models with RMSNorm and SiLU-based MLP (Feed-Forward Network) blocks, but they differ in their internal dimensions and attention configurations.

Llama-3B contains 28 transformer layers with a hidden dimension of 3,072 and an intermediate MLP dimension of 8,192. Its attention module uses 24 query heads and 8 key-value heads. The Q and O projections operate over 3,072-dimensional representations, while the K and V projections produce 1,024-dimensional outputs. The Gate and Up projections expand the hidden representation to 8,192 dimensions. Its LM head maps the final 3,072-dimensional representation to a vocabulary of 128,256 tokens.

Qwen-4B contains 36 transformer layers with a hidden dimension of 2,560 and an intermediate MLP dimension of 9,728. It uses 32 query heads and 8 key-value heads. Its Q projection produces a 4,096-dimensional representation, while K and V each produce 1,024 dimensions, and the O projection maps the 4,096-dimensional attention output back to the 2,560-dimensional hidden representation. The Gate and Up projections expand the representation to 9,728 dimensions, and the LM head maps the final hidden representation to a vocabulary of 151,936 tokens.

6.5 The Case Against the “Down” Linear Projection

The “Down” linear projection described in Section 3.2 and depicted in Figure 2 is also a linear projection involving the matrix multiplication W_{down} . The input to this component is the output of the activation function (SiLU). Theorem 3 shows the bound on the sensitivity of the activation function, and therefore the scale of noise that needs to be added to the input $\mathbf{x}$ to the Down component. In our experiments with **Llama-3B**, we found that offloading this component caused significant storage cost for the noise bank and floating-point errors (Section 5.5) after noise correction. This is due to two main reasons. First the input to

²¹See <https://www.qemu.org/docs/master/specs/ivshmem-spec.html>

²²<https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>

this component has dimension (also called intermediate dimension) equal to 8,192 ($d_{\#}$), which is significantly larger than the model’s embedding dimension (hidden dimension) $d = 3,072$. Furthermore, substituting this dimension size in the global sensitivity bound of Theorem 3 in d and the norms of the matrices W_{gate} and W_{up} yields a value much higher than the global sensitivity bounds for other components. The net result is that the noise bank has to store noise of larger scale and size, as well as significant amplification of floating-point errors after noise correction. The latter means that there is a significant mismatch between $W\mathbf{x}$ and $W(\mathbf{x} + \boldsymbol{\eta}) - W\boldsymbol{\eta}$, where the latter is calculated via offloading. This error is then carried over to other components of the LLM, resulting in a drastically different output.

One way to resolve this issue is to perform any computation from this component in 64-bit floating-point arithmetic. The reason why this would work is evident from Eq. (27). In the IEEE 754 binary64 format, also called double-precision floating-point, we have precision $p = 53$, meaning that $u = 2^{-53}$ from Eq. (16). Consequently, γ_d from Definition 6 is $\gamma_d \approx d \cdot 2^{-53}$, assuming $du \ll 0$, as should be the case. On the other hand, in the IEEE 754 binary32 format, also called single-precision floating-point, we have $p = 24$, meaning that $u = 2^{-24}$, and $\gamma_d \approx d \cdot 2^{-24}$ with the same assumption. The ratio of this to the former is approximately $2^{29} \approx 5.38 \times 10^8$. Thus the bound in Eq. (27) in the 64-bit floating-point is many orders of magnitude smaller than smaller precision formats, and hence it can solve the floating-point error amplification issue with noise correction. However, using 64-bit precision for the Down component significantly increases the noise bank size, as each stored value requires more memory. The higher precision also increases memory consumption, processing time, and the amount of data transferred through shared memory. Consequently, outsourcing this operation provides diminishing returns. We therefore retain the Down component inside the TEE.

7 Results

We present our results in the following order. We first compare LLM inference time of our split-inference scheme against single-processor inference. We then analyze the similarity of the generated response through our scheme to the non-partitioned setting. This is followed by comparison with a cryptographic variant of our protocol based on [34], and finally effectiveness of the prompt reconstruction attack on our scheme.

7.1 Inference Time

We begin by defining a few configurations to compare inference times, shown in Table 1. The CPU and GPU configurations serve as baselines for LLM inference at the CPU and the GPU, respectively. The TDX configuration aims to demonstrate the difference in computation time on a CPU-enabled TDX against the CPU and GPU. The last two configurations, i.e., EP# and EPN implement our split-inference with differentially private noise of a constant scale (e.g., $\epsilon = 1$) per token versus $\epsilon = 1/\sqrt{n}$ per token, respectively, for a total of n tokens. The configuration T3 is a vanilla split-inference configuration. Its purpose is to assess whether the masking and correction steps add additional overhead as compared to simply outsourcing noiseless inputs and receiving noise-free outputs from the GPU.

CPU	Under this configuration, entire LLM inference is performed by the CPU of the host.
TDX	Entire LLM inference is performed within the TDX guest, using the CPU of the host machine. The difference is that, in the TDX configuration, the computation and the guest’s memory are protected by Intel TDX.
GPU	Entire LLM inference is performed by the GPU of the host.
T3	Split-inference between the TDX guest and the host’s GPU but without masking and correction.
EP#	Split-inference between the TDX guest and the host’s GPU with constant noise scale ϵ . Over all n tokens of the prompt, this amounts to $\sqrt{n}\epsilon$ -Gaussian differential privacy (see Section 5.4). We use different constant values of ϵ for this configuration.
EPN	Split-inference between the TDX guest and the host’s GPU with noise of scale $\epsilon = 1/\sqrt{n}$. Over all n tokens of the prompt, this amounts to $\sqrt{n}\epsilon = 1$ -Gaussian differential privacy.

Table 1: The various configurations to compare LLM inference times.

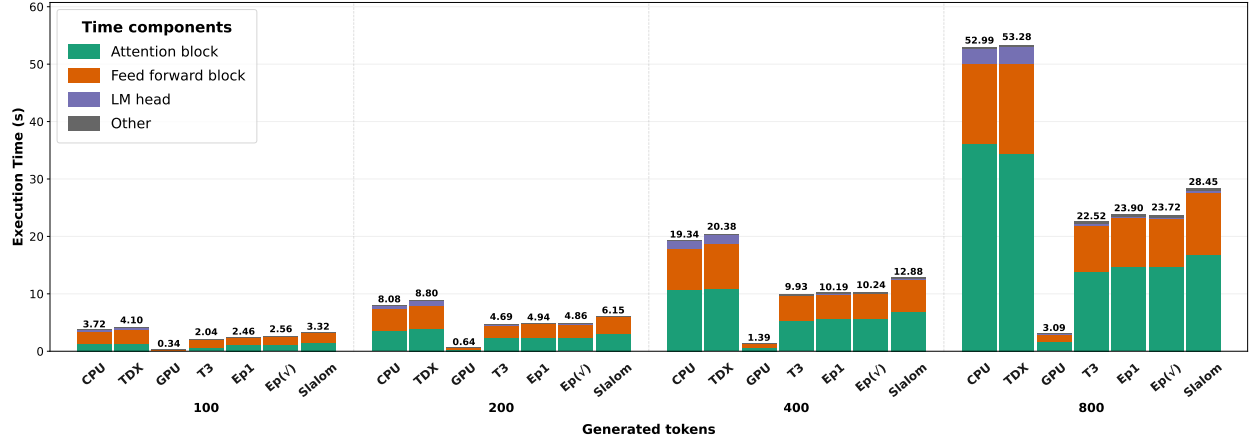


Figure 4: Execution time comparison for Llama-3B across CPU-only, full-TDX, direct-GPU, T3 (no-noise split), the noised configurations EP1 and EP($1/\sqrt{n}$), and an adapted Slalom baseline using single-precision (FP32) outsourced computation.

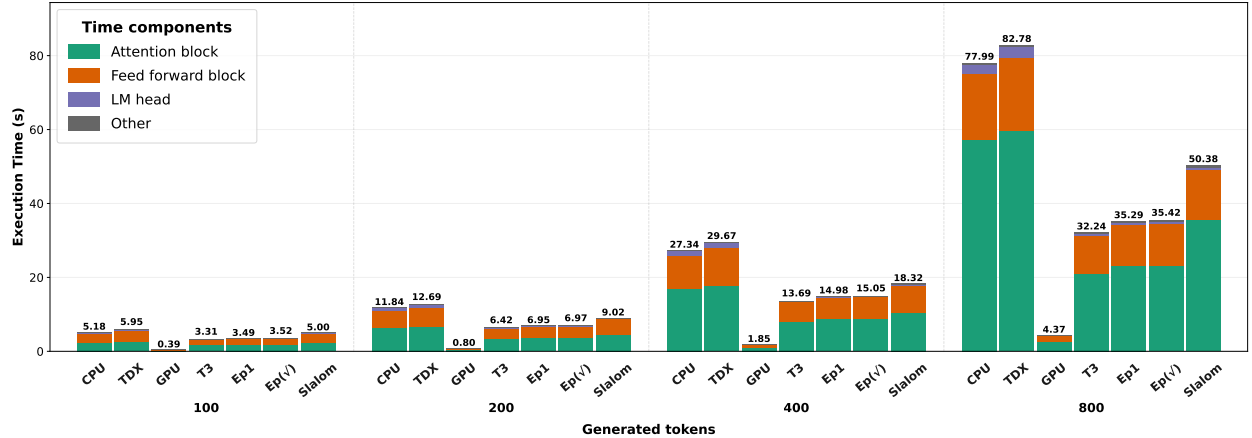


Figure 5: Execution time comparison for Qwen-4B across CPU-only, full-TDX, direct-GPU, T3 (no-noise split), the noised configurations EP1 and EP($1/\sqrt{n}$), and an adapted Slalom baseline using single-precision (FP32) outsourced computation.

For each configuration, we use prompts selected from the Databricks Dolly dataset [8]²³ with an average input length of approximately 100 tokens. This is a human-generated instruction-response dataset suitable for training instruction-following LLMs. We evaluate 45 prompts in total using a batch size of 15, resulting in three batches per configuration. Batching allows multiple prompts to be processed together in a single inference run, improving computational efficiency. In our split TDX-GPU setting, batching also reduces the number of separate data transfers and GPU calls by processing multiple prompts together. Each configuration is executed once over the same set of prompts, and we report the average inference time per prompt. To evaluate how the number of generated tokens affects inference time, we define four output-token limits of 100, 200, 400, and 800 tokens. For each setting, Llama-3B and Qwen-4B are configured to generate exactly the number of new tokens specified by the corresponding token limit. Figure 4 shows the results for all configurations for Llama-3B.

The TDX configuration is only marginally slower than the CPU configuration which shows that the trusted TDX guest only adds slight overhead over inference done on the untrusted CPU. This overhead is expected because TDX relies on hardware assisted memory protection and encryption to isolate the guest

²³Available at <https://huggingface.co/datasets/databricks/databricks-dolly-15k>

from the host [36]. Also, note that as the number of generated tokens increases, inference time increases non-proportionally, showing that the time taken is not linear in the number of tokens generated. This is because tokens are generated sequentially, and each newly generated token increases the context that subsequent attention operations must process.

Both the CPU and TDX configurations are many orders of magnitude slower than the untrusted GPU which only takes 0.34, 0.64, 1.39, and 3.09 seconds per prompt for the four token-generation sizes. This motivates the need to split inference between the TDX and the GPU to speed up inference time. The EP# configuration with $\epsilon = 1$, which we call EP1, takes times of 2.46, 4.94, 10.19, whereas the EPN configuration achieves 23.90 seconds, and 2.56, 4.86, 10.24, and 23.72 seconds, over the four token-generation sizes. Specifically, over larger token sizes, we achieve more than double the speedup using the split-configuration, e.g., 23.90 seconds with EP1 versus 53.28 seconds at the TDX for 800 tokens. The only difference between the two noisy configurations is the amount of noise added to mask the inputs, with more noise added in the EPN configuration. Both configurations therefore perform essentially the same amount of computation and communication. Interestingly, the T3 split-inference configuration, which does not add any noise nor corrects any noise, has times of 2.04, 4.69, 9.93, and 22.52 seconds. The small gap between these times and those of the two noisy configurations, EP1 and EPN, shows that noise addition and correction introduce only modest overhead.

The times in Figure 4 are divided into times taken by the attention block (linear operations Q , K , V , and O), the feed-forward block (operations Gate and Up), the LM head, and all other components. Most of the performance improvement over the TDX configuration comes by offloading the attention and feed-forward blocks. This justifies the need to offload these components to the GPU. The language-model head maps the final hidden representation to the model vocabulary. For example, Llama-3B has a vocabulary of 128,256 tokens, while Qwen-4B has 151,936 tokens. A larger vocabulary increases the size of the language-model head projection and therefore the amount of computation required by this operation. We can see that offloading the language-model head also results in a net improvement in time, although less so than the other components. Offloading these components to the GPU therefore reduces the amount of computation performed inside TDX and allows the GPU to handle the more expensive operations in parallel. The remaining operations, including normalization, residual connections, attention computation, non-linear feed forward network operations, and the Down projection, are performed inside TDX. These operations have minimal impact on the overall time (see the legend “Other” in Figure 4).

To see if our results are replicated over other LLMs, we also did the same experiment on Qwen-4B, with the results shown in Figure 5. The trend is the same as that of Llama-3B, except now there is a bigger difference between the TDX and noisy configurations. For example, for 800 generated tokens, the TDX configuration takes almost 83 seconds whereas EP1 and EPN take up to 36 seconds only.

7.2 Inference Time Comparison with Slalom

The main inspiration of our work comes from the closest existing system to ours, Slalom [34]. Slalom was proposed as a system to split neural network inference between a CPU-based TEE and an untrusted GPU, just like ours. In its original design, the TEE is an Intel Software Guard Extension (SGX) enclave. Just like our system, expensive linear operations are assigned to the GPU, and since the GPU is not trusted, Slalom masks the values before outsourcing them and removes the corresponding correction after the result is returned.

However, there are a few main differences between their work and ours. First, Slalom uses Intel SGX as the trusted environment, whereas our implementation runs inside an Intel TDX VM. This is mainly due to the fact that Intel TDX is a newer technology. Indeed, their system can easily be ported into an Intel TDX VM. Secondly, their system was designed for a conventional deep neural network (DNN) containing multiple layers, and not specifically for LLMs. LLMs contain many repeated transformer layers with large linear projections, and these projections can operate on substantially higher-dimensional inputs and outputs than those considered in the original Slalom evaluation. Applying the Slalom masking mechanism to the outsourced linear operations in an LLM therefore requires more computation to generate the corresponding corrections and more memory to store the precomputed masks and corrections.

The third and major difference is how they mask the input to the offloaded linear operations. They work in the finite field $\mathbb{Z}_q$ modulo a prime q . The d -dimensional mask $\boldsymbol{\eta}$ is then a vector each element of

which is a random element from the field $\mathbb{Z}_q$. The input $\mathbf{x} \in \mathbb{R}^d$ is then *quantized* into an integer vector $\mathbf{x}_{\text{qt}} \in \mathbb{Z}_q^d$ of the same dimension via fixed-point representation. The masked input is then $\tilde{\mathbf{x}}_{\text{qt}} = \mathbf{x}_{\text{qt}} + \boldsymbol{\eta} \pmod{q}$. Likewise, the weight matrix W is quantized as W_{qt} into a matrix of elements from $\mathbb{Z}_q$. The GPU computes $W_{\text{qt}}\tilde{\mathbf{x}}_{\text{qt}} = W_{\text{qt}}(\mathbf{x}_{\text{qt}} + \boldsymbol{\eta}) \pmod{q}$, and sends the result back to the TEE, who then corrects the noise by computing $W_{\text{qt}}(\mathbf{x}_{\text{qt}} + \boldsymbol{\eta}) - W_{\text{qt}}\boldsymbol{\eta}$. The result can then be de-quantized back to a vector in $\mathbb{R}^d$ (or more precisely a vector of floating-point equivalents) to continue operations in the original domain inside the TEE. It is important to remark that Slalom also checks the integrity of the result of matrix multiplication from the GPU through Freivalds’ algorithm [14], hence another reason to work within the field $\mathbb{Z}_q$. However, we omit this part as we do not consider the GPU to be malicious in our threat model.

To compare the inference times and accuracy of our split-inference mechanism based on differential privacy against that of Slalom’s, based on cryptography (essentially a one-time pad), we adapt Slalom’s masking mechanism to the same LLM components outsourced in our system. Thus, the (Q, K, V) projections, the O projection, Gate/Up projections, and LM head are executed on the untrusted GPU, while the remaining operations stay inside the TDX guest. Before offloading an input $\mathbf{x}$ to the GPU, we quantize it and then mask it using a fresh random vector using the Slalom protocol outlined above. The weight matrices at the GPU are also quantized. The GPU does the matrix computation in the finite field $\mathbb{Z}_q$. The returned result is then corrected using precomputed (quantized) correction as shown above. The recovered result is then de-quantized back to floating point before the remaining operations continue inside the TDX.

We use a fresh, single-use mask for each outsourced computation, following the confidentiality mechanism used by Slalom. Following Slalom’s implementation, we use the prime modulus $q = 2^{23} + 2^{21} + 7 = 10,485,767$ and 8-bit quantization for both input tensor and model weights.²⁴ Figure 4 shows that the run-time through Slalom is slower than our method. Specifically, for 400 generated tokens Slalom is 2 seconds slower, whereas for 800 generated tokens it is almost 5 seconds slower than our scheme. Looking at Figure 5, Slalom is almost 15 seconds slower than our scheme on **Qwen-4B**. Inference time is just part of the comparison. As we shall show next, one of the drawbacks of Slalom is that we cannot tweak its accuracy as easily as our scheme by changing the value of ϵ .

7.3 Similarity of Generated Text

For our scheme to be viable, the split-inference architecture should not degrade the quality of the response from the LLM. To test this, we compare the outputs generated by the original and the split-inference models under greedy decoding. Greedy decoding outputs the next token with the highest score (logit) produced by the language model head.

7.3.1 Similarity Score

We evaluate 100 prompts with input lengths between 100 and 250 tokens, selected from the Databricks Dolly dataset, which cover a range of instruction-following tasks such as factual question-answering and context-based queries. For each prompt, we generate exactly 200 new tokens using greedy decoding. We evaluate fixed privacy levels $\epsilon \in \{0.5, 1, 5, 10, 15\}$ for the EP# configuration and additionally evaluate the EPN configuration with $\epsilon = 1/\sqrt{n}$, where n denotes the number of tokens in the input prompt. We compare the responses generated by the protected split-inference configurations against the GPU configuration as the baseline. The GPU configuration produces the same outputs as the CPU, TDX, and T3 configurations under greedy decoding. We therefore use the GPU output as the reference and evaluate EP# and EPN configurations against it.

To measure how closely the responses generated by the protected split-inference model match those of the original model, we use the following metrics: exact output match, token match rate, BLEU, ROUGE-L, and semantic similarity. Exact output match reports the percentage of prompts for which the complete generated token sequence is identical. Token match rate in contrast gives the percentage of the generated tokens that were the same. BLEU measures similarity based on overlapping n-grams [30], while ROUGE-L measures the longest common subsequence between the responses [25]. Semantic similarity measures if the overall meaning of the two responses is preserved by computing cosine similarity between their sentence

²⁴We use the following implementation of Slalom: <https://github.com/ftramer/slalom>.

Table 2: Output comparison between the original model and the split-noised model under greedy decoding. Each setting contains 100 prompts with 200 generated tokens per prompt.

ϵ	Exact-match	Token match	BLEU	ROUGE-L	Semantic similarity
$1/\sqrt{n}$	10.00%	42.55%	64.49%	69.92%	95.56%
0.5	67.00%	85.43%	91.59%	93.26%	99.36%
1	84.00%	91.49%	94.44%	95.38%	99.67%
5	98.00%	99.41%	99.57%	99.59%	99.96%
10	100.00%	100.00%	100.00%	100.00%	100.00%
15	100.00%	100.00%	100.00%	100.00%	100.00%
Slalom	0.00%	20.22%	43.82%	52.28%	93.11%

embeddings using the `all-mpnet-base-v2` sentence-transformer model.²⁵

Table 2 shows that the output generated by the EP# configuration becomes more stable as ϵ increases. At $\epsilon = 0.5$, 33% of the generated responses diverge from the original output. This decreases to 16% at $\epsilon = 1$ and only 2% at $\epsilon = 5$. At $\epsilon = 10$ and $\epsilon = 15$, no deviations are observed and all 100 generated responses match the original outputs exactly. The token match rate, BLEU, and ROUGE-L scores follow the same trend, improving as ϵ increases. Noticeably, across all ϵ values the semantic similarity of the responses from our scheme remains particularly high, ranging from 99.36% at $\epsilon = 0.5$ to 100% at $\epsilon \geq 10$. This shows that even when some generated tokens differ, the responses generally retain the same meaning as the original output.

The prompt-length-dependent setting EPN results in stronger perturbation, with an average ϵ of 0.0773 across the evaluated prompts. As a result, the generated responses differ more often from the original outputs. Only 10% of the responses match exactly, while the token match rate decreases to 42.55%. BLEU and ROUGE-L are 64.49% and 69.92%, respectively. However, semantic similarity remains high at 95.56%, suggesting that much of the overall meaning of the responses is still preserved despite the larger token-level differences.

Comparison with Slalom. Compared with our configurations, Slalom shows substantially larger output divergence. None of the 100 Slalom responses match the GPU baseline exactly and results in a token match rate of 20.22%, with BLEU and ROUGE-L scores of 43.82% and 52.28%, respectively. Despite this larger token-level divergence, the semantic similarity remains relatively high at 93.11%, indicating that the generated responses often preserve the main meaning even when the wording differs considerably from the GPU baseline. The advantage of our scheme is that we can tweak ϵ for the EP# configuration to achieve a desired level of accuracy. This tunability is absent in Slalom, as the decrease in accuracy is an artifact of quantization.

Examples of Generated Prompts. Table 3 illustrates these differences using a complete example, showing how the GPU baseline, EPN, EP1 and Slalom responses diverge over the full 200-token generation.

7.3.2 Floating-Point Errors

The main reason for divergent responses in our scheme is the accumulation of floating-point errors during noise correction as discussed in Section 5.5. To measure this effect, we compute the numerical error specified in Eq. (15), together with the error bound in Eq. (27), for individual output elements of all outsourced linear operators, namely the Q , K , V , O , Gate, and Up projections, as well as the final LM-head projection.

We use 100 prompts from the Dolly dataset with input lengths between 100 and 250 tokens and capture the actual inputs to each outsourced operator during inference. We use both the EP# and EPN configurations with different values of ϵ for the former. For each evaluated ϵ , this gives 169 operator instances in total: six outsourced projections across each of the 28 transformer layers, together with the final LM head. For each operator instance, we perform 1,000 trials. In each trial, we randomly select one of its captured input vectors

²⁵See <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

Table 3: Example prompts and generated responses for the GPU baseline, EPN, and Slalom. Each response is limited to 200 generated tokens and therefore ends at the generation cap. Differences from the GPU baseline are shown in **red** for both EPN and Slalom.

Prompt	When was the first Reading railway station opened? <i>Context:</i> Reading railway station is a major transport hub in Reading, Berkshire, England. It is on the northern edge of the town centre, near the main retail and commercial areas and the River Thames, 36 miles (58 km) from London Paddington. The first Reading station was opened on 30 March 1840 as the temporary western terminus of the original line of the Great Western Railway (GWR). Reading is the ninth-busiest station in the UK outside London and the second busiest interchange station outside London with over 3.8 million passengers changing trains at the station annually.
GPU	The first Reading railway station was opened on 30 March 1840. It was the temporary western terminus of the original line of the Great Western Railway (GWR). The station has since undergone several expansions and renovations, but this date marks its initial opening. The station has become a major transport hub in Reading, serving over 3.8 million passengers annually. It is also a key interchange station, with connections to other parts of the UK. The station’s location near the River Thames and the town centre makes it a significant transportation hub in the region. The station’s history dates back to 1840, and it has played a crucial role in the development of Reading as a major town in Berkshire, England. The station has undergone several expansions and renovations over the years, but its initial opening in 1840 marked the beginning of its growth as a major transport hub. The station’s strategic location near the River Thames and the town centre has made it an essential transportation link for the region
EP1	The first Reading railway station was opened on 30 March 1840. It was the temporary western terminus of the original line of the Great Western Railway (GWR). The station has since undergone several expansions and renovations, but this date marks its initial opening. The station has become a major transport hub in Reading, serving over 3.8 million passengers annually. It is also a key interchange station, with connections to other parts of the UK. The station’s location near the River Thames and the town centre makes it a significant transportation hub in the region. The station’s history dates back to 1840, and it has played a crucial role in the development of Reading as a major town in Berkshire, England. The station has undergone several expansions and renovations over the years, but its initial opening in 1840 marked the beginning of its growth as a major transport hub. The station’s strategic location near the River Thames and the town centre has made it an essential transportation link for the region
EPN	The first Reading railway station was opened on 30 March 1840. It was the temporary western terminus of the original line of the Great Western Railway (GWR). The station has since undergone several expansions and renovations, but this date marks its initial opening. The station has become a major transport hub in Reading, serving over 3.8 million passengers annually. It is also a significant interchange station, with connections to other lines and trains . The station’s location near the River Thames and the town centre makes it a convenient and accessible point for commuters and travelers . The station’s history dates back to the 19th century , and it has played a vital role in the development of Reading as a major transportation hub . The station’s current status as the ninth-busiest station in the UK outside London and the second busiest interchange station outside London is a testament to its importance in the region. The station’s growth and development over the years have made it an essential part of the transportation infrastructure in
Slalom	The first Reading railway station was opened on 30 March 1840. It was the temporary western terminus of the original line of the Great Western Railway (GWR). This was over 183 years ago. The station has since undergone several expansions and renovations, but its initial opening date remains 30 March 1840. It’s worth noting that the station has been in operation for nearly two centuries, and it continues to be a major transport hub in Reading, serving over 3.8 million passengers annually. Its strategic location near the River Thames and the town centre makes it an essential transportation link to London and other parts of the country. The station’s history and evolution over the years have played a significant role in the development of Reading and the surrounding region. It has been an integral part of the town’s growth and transformation into the thriving city it is today. The station’s rich history and ongoing importance in the transportation network make it a notable landmark in the UK, with

and one output row from its corresponding weight matrix, and apply Gaussian noise using the operator-specific global sensitivity. This results in 169,000 evaluated output elements for each ϵ . We report the mean

Table 4: Numerical-error results across the evaluated ϵ values.

ϵ	Mean true error	Mean bound	Bound/error ratio	Bound violation [†] (%)
$1/\sqrt{n}$	1.52×10^{-3}	6.53×10^{-3}	$4.30\times$	9.93%
0.5	2.26×10^{-4}	9.64×10^{-4}	$4.27\times$	10.03
1	1.13×10^{-4}	4.83×10^{-4}	$4.28\times$	9.89
3	3.75×10^{-5}	1.61×10^{-4}	$4.30\times$	9.83
5	2.26×10^{-5}	9.67×10^{-5}	$4.29\times$	9.70
7	1.61×10^{-5}	6.91×10^{-5}	$4.29\times$	9.68
10	1.12×10^{-5}	4.86×10^{-5}	$4.33\times$	9.51
15	7.48×10^{-6}	3.25×10^{-5}	$4.35\times$	9.25

[†]Fraction of evaluated output elements for which the measured numerical error exceeds the computed bound.

measured numerical error and mean computed bound over all 169,000 evaluations for each ϵ . For the bound, we use numerical precision $p = 24$, corresponding to the IEEE binary32 format.

Table 4 reports the mean true error and the corresponding mean error bound. Both decrease as ϵ increases, following the expected reduction in masking noise. The mean bound is approximately 4 times larger than the mean observed error, indicating that it is conservative on average. However, the bound is a good indicator of the amount of error caused by a given value of ϵ . This can then be used to choose a suitable value of ϵ to balance privacy and utility. Note that computing the bound is a lot easier, and does not require running end-to-end inference. We can simply sample expected inputs $\mathbf{x}$ and generate the mask $\boldsymbol{\eta}$ to compute the bound, given the weights of the matrix W . We note that there were some instances where the actual error was higher than the computed bound, as indicated by the last column in Table 4. This is because our bound is derived based on a particular method of computing dot products. Machine learning libraries might use certain optimizations in some cases that are not reflected in our simplified calculation.

How Much Numerical Error is Too Much? To understand how much numerical error the model can tolerate before output quality starts to degrade, we performed an additional controlled-error experiment. We used 100 prompts from the same Dolly dataset and evaluated all prompts at each controlled error level. For each outsourced operator, we first measured the numerical error that is naturally created by the masking and unmasking process under EP1. This error is the difference between the clean projection output under T3 and the corrected output recovered after masking, and is computed separately for every operator invocation rather than using a single fixed error value. We then preserved the error pattern observed for each operator call and changed only its magnitude. This allowed us to gradually increase the numerical error seen by the model while keeping the structure of the naturally occurring masking error.

Table 5 reports the results of this experiment. The T3 setting represents clean split inference without masking and therefore has zero numerical error. The EP1 setting uses the masking mechanism without any adjustment to the resulting error. Its mean absolute error of 2.06×10^{-4} therefore corresponds to the naturally occurring error produced by the masking and correction process. For the controlled-error settings, the EP1 error pattern is preserved for each operator call and its magnitude is adjusted to the mean absolute error levels shown in Table 5, ranging from 1.00×10^{-4} to 3.00×10^{-2} . These values therefore represent controlled error levels rather than naturally occurring errors. The generated output at each error level is compared with the corresponding T3 output using semantic similarity. We compute this using the `all-mpnet-base-v2` Sentence-Transformer model and cosine similarity, reported as a percentage.

The results show that the natural numerical error introduced under EP1 has little effect on generation quality. At a mean absolute error of 2.06×10^{-4} , semantic similarity remains at 99.20%. When the mean absolute error is increased to 1.00×10^{-3} , similarity remains at 94.48%, while an error of 1.00×10^{-2} reduces it to 82.99%. A much larger drop is observed at higher error levels, with semantic similarity decreasing to 65.36% at 1.40×10^{-2} , 43.04% at 1.60×10^{-2} , and 9.91% at 2.00×10^{-2} . These results show how generation quality changes as the numerical error increases and provide a practical reference for interpreting the error values reported in Table 4. More specifically, since the error changes with the value of ϵ in the EP# configuration, we can check the error bound from Eq. (27). If the bound is above 10^{-4} , the table suggests that the corresponding computations will degrade generation quality. It is the feature of our scheme that

Table 5: Effect of controlled numerical error on generation quality.

Setting	Mean absolute error	Semantic similarity
T3	0	100.00%
EP1	2.06×10^{-4}	99.20%
Controlled error*	1.00×10^{-4}	99.49%
	3.00×10^{-4}	97.98%
	1.00×10^{-3}	94.48%
	3.00×10^{-3}	90.35%
	1.00×10^{-2}	82.99%
	1.20×10^{-2}	74.83%
	1.40×10^{-2}	65.36%
	1.60×10^{-2}	43.04%
	1.80×10^{-2}	20.69%
	2.00×10^{-2}	9.91%
	3.00×10^{-2}	3.07%

*EP1 error scaled to the specified mean absolute error.

this error can be controlled by choosing a suitable value of ϵ balancing privacy and accuracy.

Token Divergence is Much More Complex. The token-level traces show that output divergence is not determined by the numerical error magnitude alone. A token flip occurs when the correction-induced error changes the ordering of the highest scoring logits. When the baseline model assigns very similar scores to its top two candidates, even a small numerical error can change which token is selected. In autoregressive generation, each newly generated token becomes part of the context used to predict the next token; this evolving context is referred to as the autoregressive state. Therefore, once the first token changes, the subsequent tokens may follow a different generation path. Having said that, if the logits of top few tokens are extremely close, then replacing one with another will not cause significant change in the model’s utility. In the evaluated prompts, increasing ϵ reduces both the numerical error and the frequency of output divergence, with no divergence observed at $\epsilon \geq 10$. Although the numerical error alone does not determine whether a token will change, it can still be used as a practical guide to output stability. In our experiments, smaller mean true errors were generally associated with fewer divergent outputs and higher token and semantic similarity. Therefore, the approximation error provides a useful indicator of whether masking is likely to negatively impact the utility of the generated output.

7.4 Prompt Reconstruction Attack

Two questions may naturally arise in the reader’s mind. First, since the untrusted GPU is never given the user prompt in the clear, and only derived inputs to the offloaded components, is it necessary to mask these inputs? Indeed, if it is difficult to reconstruct the prompt from these inputs, then adding differentially private noise is pointless. Secondly, if it does turn out that masking the input is necessary, how much protection does differential privacy provide in practice? The interpretation of our privacy protection is that the GPU, who is given a single input to a linear layer, is not able to distinguish with high probability between two prompts of length n . However, we also mentioned that the GPU is given multiple inputs derived from the same prompt. Thus to justify using the same ϵ for all inputs derived from a prompt, we need to test the strength of an attack in practice.

Recall that the GPU is honest-but-curious: it performs all computations as dictated, yet it can use any information provided to it to infer the user’s prompt. This information includes all (masked) inputs to the offloaded linear components, as well as the architecture and weights of the learned model (the service provider knows its own model!). However, any processing done at the TDX guest, including data in its private memory, is not available to the GPU.

To model this, we use a prompt reconstruction attack inspired by the generative embedding inversion attack of Li et al. [24], which trains a generative decoder to recover an input sequence from its sentence embedding. Unlike a pooled sentence embedding, the inputs in our attack contain token-level representations produced during model execution which may retain substantial lexical and semantic information about the original prompt. This attack can be launched by the GPU (or an attacker that sees information to-and-from the GPU) by training an attack model on a publicly available text dataset and a local copy of the LLM to learn the relationship between prompts and the offloaded inputs.

7.4.1 Training the Reconstruction Attacker

We use the Pile dataset [15],²⁶ to train our prompt reconstruction attack, which is a diverse dataset for training large language models. Specifically, we use 2,000 prompts divided into 1,800 training prompts and 200 validation prompts. Each prompt is passed through the LLM, and the inputs exposed at a selected outsourced linear operator are recorded together with the original prompt. For the noise-aware attack model, the corresponding noise mechanism and privacy setting (value of ϵ) are applied to the captured inputs before they are used for training. These intermediate input-prompt pairs are then used to train a generative reconstruction model. The reconstruction attacker uses a pretrained Llama-3B model to reconstruct the original prompt. The attacker receives an intermediate representation captured from the first transformer layer (Layer 0) of the victim model. In our experiment, the attacker can use the intermediate representations exposed at any outsourced operator as the input. The dimensionality of the captured representation depends on the selected operator and capture point. For example, the input to the (Q, K, V) projections in Layer 0 has the dimension of 3072.

The captured hidden states cannot be passed directly to the attacker model in the same way as normal token embeddings. Although they may have the same dimension, they represent internal features produced by the victim model rather than the attacker model’s input-embedding space. We therefore use a small trainable projection network, which acts as a learned mapping between these two representations. Here, the projection network is a small neural network placed between the captured tensor and the attacker LLM.

The projection network first expands each captured vector into a higher-dimensional space so that it has more capacity to transform and reorganize the captured information. In our implementation, this intermediate space is four times the input dimension. A SiLU activation is then applied to introduce a nonlinear transformation. The representation is subsequently reduced back to the embedding dimension expected by the attacker model, followed by a second SiLU activation. Applying SiLU after both the expansion and reduction layers allows the projection network to capture more complex nonlinear relationships between the captured representation and the attacker model’s embedding space. Finally, RMSNorm is applied to keep the projected representation on a stable scale before it is passed to the attacker model.

The output of the projection network is then used as a *soft prefix* for the attacker model. Here, a soft prefix is a sequence of continuous embedding vectors produced by the projection network. It provides the initial context for generation, similar to a normal token prefix. Unlike a normal text prefix, it is not created from input token IDs. Instead it provides the attacker with the starting context needed to reconstruct the original prompt.

During training, the attacker is given the projected representation along with the original prompt as the target output. We use teacher forcing, where the correct preceding prompt tokens are provided to the model while it learns to predict the next token, rather than relying on its own previous predictions [38]. This helps the attacker learn how to reconstruct the prompt token by token from the information contained in the captured representation. The target sequence also includes an end-of-sequence (EOS) token so that the model learns when to stop generating.

To keep the reconstruction attacker lightweight, most of the pretrained attacker model is kept frozen during training. Instead of updating all model parameters, we use Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning method that introduces a small number of trainable parameters into selected layers while leaving the original model weights unchanged [19]. This allows the language model to adapt to the prompt-reconstruction task without requiring full model fine-tuning. At the same time, the projection network is trained to transform the captured intermediate representation into a soft prefix compatible with the attacker model’s embedding space. The projection network and LoRA adapters are trained jointly, allowing

²⁶Available here: <https://huggingface.co/datasets/ElleutherAI/pile>.

the attacker to learn how to interpret the captured representation and reconstruct the original prompt while updating only a small fraction of the overall model parameters. The reconstruction model is trained for five epochs. The maximum target length used during teacher forcing is increased gradually, allowing the model to first learn shorter portions at the beginning of the prompt before being trained on the complete target sequence. After each epoch, the model is evaluated on a held-out validation set, and a checkpoint is saved. The checkpoint with the lowest validation loss is selected as the attacker model used in the final evaluation. The training of the reconstruction model is depicted in Figure 6.

After training, the attacker is evaluated using previously unseen prompts from the Databricks Dolly dataset. The inputs exposed to the GPU from unseen prompts are recorded and provided to the trained reconstruction model. The generated sequence is taken as the reconstructed prompt.

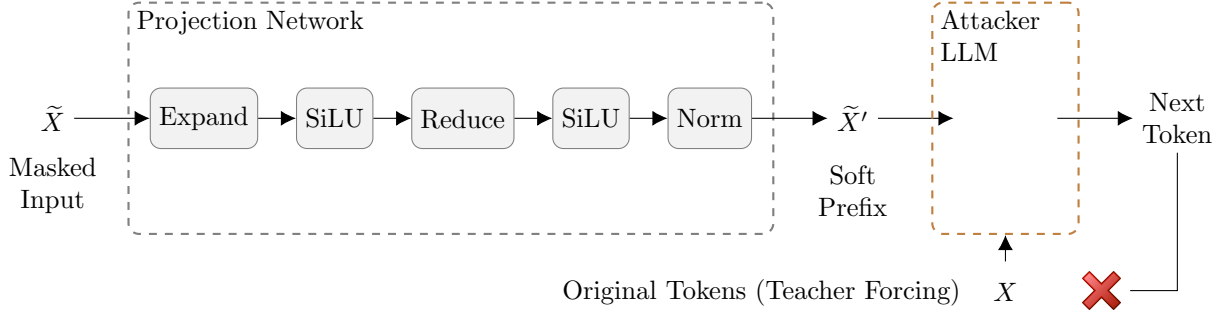


Figure 6: The training phase of the prompt reconstruction model based on [24]. The masked input $\tilde{X}$ is given as input to a projection network which creates a soft prefix $\tilde{X}'$. This is given as input to the attacker’s LLM (Llama-3B). The model is teacher forced by feeding it tokens from the original prompt X instead of its own generated tokens.

7.4.2 Attacker Settings

We evaluate the reconstruction attack under several attacker settings which vary in terms of the observed inputs and the attacker’s knowledge of the masking mechanism.

1. *Noise-free attacker*: We first consider a reconstruction model trained and evaluated on unmasked inputs. As mentioned in the beginning of this section, this setting determines if it is necessary to mask inputs to linear operations offloaded to the GPU.
2. *Noise-oblivious attacker*: In this setting, the reconstruction model from the noise-free setting is applied to masked inputs. This represents an attacker who has trained its model without knowledge of the masking mechanism and serves to highlight the need for training the attack model with auxiliary masked inputs.
3. *Noise-aware attacker*: Next, we consider an adaptive attacker who knows the masking mechanism. The attacker further trains the model by generating auxiliary training inputs using the same noise mechanism and privacy setting (value of ϵ) as the protected system. The resulting noise-aware reconstruction model is then evaluated on masked inputs collected from the real TDX-GPU execution. This setting tests whether the masking mechanism remains effective when the adversary accounts for the applied noise during training.

The noise-free, noise-oblivious and noise-aware attacks can be further divided into how many components are taken into account by the attack model.

1. *Single Component*: This attack is launched on a single component. The attack is most potent when applied on the inputs to the (Q, K, V) component (recall that each of the three matrices receive the same input). The attack can be applied to any other outsourced component. Since the (Q, K, V) component is closest to the original prompt than the ensuing outsourced components, it provides the most information about the prompt.

2. *Composite attacker*: This attack uses inputs exposed by multiple outsourced components from the same transformer layer. We combine the inputs to the (Q, K, V) projections, the O projection, and Gate/Up projections. The three inputs are aligned by token position and concatenated along the model’s dimension d . Since each input has tensor shape $[n, d]$, where n is the prompt length and d is the hidden dimension, the resulting observation has shape $[n, 3d]$. For the evaluated Llama-3B model, this corresponds to combining three $[n, 3072]$ tensors into a single $[n, 9216]$ tensor. The reconstruction model then separates the combined observation into its (Q, K, V) , the O projection, and Gate/Up components. Each component is processed by an independent projection branch, after which the projected representations (see Figure 6) are fused into a single soft prefix. This soft prefix is used to generate one reconstruction of the original prompt. The same procedure is used for both noise-free and masked settings. In the masked setting, each operator input is independently noised using its corresponding global sensitivity before the three input tensors are combined. This represents an adaptive attacker that knows the masking mechanism and trains on auxiliary observations generated using the same noise procedure and privacy setting as the protected system.

A couple of remarks are in order. First, we did not include the LM head component in the composite attack as it did not show any improvement over the existing composition of the other three components. The reason being that the LM head is at the very end of the inference pipeline, after all transformer layers, and therefore contains the least amount of information about the prompt. Secondly, we limit the attack to Layer 0 of the transformer. This is because the inputs at this layer are the closest to the original prompt, and have undergone the least amount of transformation. Adding more layers does not substantially increase the strength of the attack without increasing its training complexity.

7.4.3 Reconstruction Attack Results

To compare the reconstructed prompt with the original prompt we use the following metrics: token precision, token recall, token F1-score, and semantic similarity. Token precision measures the proportion of generated tokens that also occur in the original prompt, while token recall measures the proportion of original-prompt tokens recovered by the attacker. Token F1-score provides a balanced summary of precision and recall. Semantic similarity is computed as the cosine similarity between sentence embeddings of the original and reconstructed prompts. We evaluate these metrics on 500 prompts from the Dolly dataset, with prompt lengths ranging from 100 to 300 tokens.

Table 6 presents the Layer 0 reconstruction results for the various attacker settings. In the noise-free setting, given inputs to the (Q, K, V) component, the attack model is able to retrieve the original prompt with very high accuracy, with token precision, recall, and F1-score all around 70%, and a semantic similarity of 79.87%. Moreover, using the LM-head input reduces overall reconstruction performance, although recall and semantic similarity remain relatively high. This is likely related to the LM-head representation is produced later in the inference pipeline and is therefore less directly related to the original prompt than the (Q, K, V) input. Since the (Q, K, V) input gives stronger and more consistent results across the metrics, we use it as the single-component attack setting in the rest of the analysis. What is also evident from the table is that the composite attacker which receives inputs to three components, (Q, K, V) , O projection and Gate/Up projections does not fare significantly better than the attack with a single component. This shows that the components which come later in the inference pipeline are not informative enough to increase the attack’s accuracy. All in all, the results show that without masking the inputs to the offloaded layers, the attacker (GPU) can infer significant information about the prompt. An example comparing an original prompt with its reconstruction is provided in Appendix C.

In comparison, both the single component and composite attacker on noisy inputs are unable to match the success rate of the noise-free attack. Specifically, the semantic similarity of the reconstructed prompts remains around 5%. Furthermore, the results are only slightly better for the noise-aware attacker versus the noise-oblivious attacker, meaning that additional knowledge of the noise mechanism in training the attack model does improve the attacker’s success, but the improvement is marginal.

Although the noise-aware composite attacker achieves a token precision of 12.72% and a token recall of 20.69%, these values do not necessarily indicate meaningful reconstruction of the original prompt. Two randomly selected prompts can still contain overlapping tokens even when they are unrelated. To examine this, we used the same 500 evaluation prompts and compared each prompt with every other prompt in the

Table 6: Prompt-reconstruction results at Layer 0 under the evaluated attacker settings.

Condition	Attacker setting	Captured operators	Token precision	Token recall	Token F1	Semantic similarity
Without noise	Noise-free	QKV only	71.77%	72.12%	70.30%	79.87%
	Noise-free	LM Head	45.40%	75.75%	55.15%	77.13
	Composite	QKV + O + Gate/Up	75.25%	69.14%	71.20%	82.19%
With noise	Noise-oblivious	QKV only	6.5%	11.06%	7.99%	5.33%
	Noise-aware (Ep1)	QKV only	13.14%	21.81%	15.91%	5.29%
	Noise-aware (Ep1)	LM head	12.63%	20.78%	15.23%	4.68%
	Composite noise-aware (Ep1)	QKV + O + Gate/Up	12.72%	20.69%	15.30%	5.23%
	Composite noise-aware (Ep1000)	QKV + O + Gate/Up	10.86%	17.98%	13.16%	6.55%
	Composite noise-aware (Ep2500)	QKV + O + Gate/Up	71.61%	55.92%	62.79%	78.41%

set, excluding self-comparisons. This produced $\binom{500}{2} = 124,750$ prompt pairs. Across these comparisons, the average token precision was 22.02%, the average token recall was 22.02%, and the average token F1 was 20.71%. These values are higher than those observed for the noised attacker. This indicates that the remaining token overlap in the noised setting is within the level that can arise naturally between unrelated prompts, rather than providing evidence of effective reconstruction of the original prompt.

To further examine how the amount of noise affects reconstruction, we also evaluated the composite noise-aware attacker at substantially larger ϵ values. Increasing ϵ reduces the amount of noise added to the outsourced representations and therefore weakens the protection. Importantly, the lower ϵ settings used in our main evaluation do not directly expose the model to the same noisy representations seen by the attacker. The untrusted GPU observes the masked operator input, while the TEE removes the corresponding precomputed correction from the returned linear result before inference continues. As a result, stronger masking can significantly reduce the information available to the attacker while still allowing the model to recover the intended computation, subject only to the numerical error introduced by floating-point masking and correction.

For each evaluated value, we followed the same noise-aware training procedure described earlier, generating the training representations using the same ϵ applied during TDX inference. Since training a separate attacker for every ϵ value is computationally expensive, we evaluated several substantially larger ϵ values rather than performing a fine-grained sweep. Table 6 shows the results for $\epsilon = 1000$ and $\epsilon = 2500$. At $\epsilon = 1000$, reconstruction remains poor, with a semantic similarity of only 6.55%, whereas at $\epsilon = 2500$ the attacker is again able to recover substantial prompt information, reaching 78.41% semantic similarity. These experiments are not intended to recommend such large ϵ values. Instead, they illustrate how reconstruction becomes possible again as the masking noise is reduced.

Can Post-Processing Recover More Prompt Information? The raw reconstructions produced by the attacker often contain repeated fragments, corrupted words, and broken sentence structure. To examine whether these outputs still contain recoverable information that is not fully reflected by the raw reconstruction metrics, we perform a separate text-restoration step on all reconstructed prompts in the evaluation set. The restoration model receives only the text produced by the attacker and is instructed to correct corrupted wording, merge repeated fragments, and improve readability without adding new information or answering the prompt. We use GPT-5.4²⁷, a highly capable general-purpose language model, for this step. Then we compare the restored text with the original prompt using the same metrics used for the initial reconstruction. The system prompt used for restoration is shown below and is designed to keep the model focused on recovering information already present in the reconstructed text.

²⁷<https://developers.openai.com/api/docs/models/gpt-5.4>

System Prompt

You are a precise text restoration assistant. The input was reconstructed from neural activations and may contain repetition, corrupted words, missing words, grammar errors, spacing errors, and incomplete fragments. Restore the entire text into the closest readable and coherent version while preserving all recoverable content and specific wording. Merge repeated fragments into one complete version and repair broken sentence structure, grammar, word order, and spacing whenever this can be done from the available context. If a short function word or grammatical connector is clearly missing, you may restore it when needed to make the sentence grammatically correct, but do not introduce new facts, entities, claims, or details that are not supported by the input. Do not summarize, shorten, or omit names, numbers, places, technical terms, questions, or contextual details. The final text must be readable, coherent, and grammatically correct while remaining as faithful as possible to the information present in the reconstructed text. Do not answer the prompt. Return only the restored text.

For the noise-free QKV reconstruction, the restoration step substantially improves the recovered text, increasing token precision from 71.77% to 89.20%, token F1 from 70.30% to 79.03%, and semantic similarity from 79.87% to 89.54%. This shows that the raw noise-free reconstruction contains additional recoverable information that is partly obscured by corrupted wording. In contrast, restoration provides no meaningful improvement for the noise-aware QKV attacker at $\epsilon = 1$. Token precision increases only slightly from 13.14% to 14.93% and token F1 slightly decreases from 15.91% to 15.76%. Semantic similarity also shows no improvement after restoration. This indicates that the noised reconstruction does not contain substantial prompt information that can be recovered through post-processing, even when a highly capable language model is used for restoration. Appendix C provides an qualitative example showing how restoration affects both the noise-free and noise-aware reconstructions.

8 Limitations and Future Directions

- Our scheme uses the fact that the current generation of TEEs does not provide hardware-isolated GPU execution. This might change in the future. However, we expect that the computational performance of such TEEs may still fall substantially short of that of unprotected state-of-the-art GPUs, particularly for large-scale LLM inference.
- Precomputing the mask and noise correction reduces inference time at the expense of storage space. Recall that the noise bank is stored in the TDX guest’s protected memory. Depending on the number of prompts, and the size of the model, this can exceed the memory specification. This drawback is similar to preprocessing in secure multiparty computation, where increased storage is required to trade off online computational time. Likewise, this issue also applies to Slalom [34]. One way to alleviate this issue is to keep unused noise in an encrypted form on the untrusted disk, and then only load and decrypt it inside the TDX when required, similar to what is done in [34]. Through a separate process, the TEE could continuously generate noise, encrypt and store it in the untrusted disk for future use.
- Our global sensitivity analysis is worst-case. In particular, it does not take into account the range of values that may be taken by embeddings from actual prompts. For instance, we use the fact that normalized vectors have a Euclidean norm of less than $\sqrt{d}$ where d is the embedding dimension. In practice, the norm may be well below this value. Thus, it may be possible to further reduce the scale of required noise, e.g., by examining local or smooth sensitivity [29] of these inputs. This is suggested by our reconstruction attack results which show that even very high values of ϵ are effective in protecting the prompt.
- Our prompt reconstruction attack may not be the most powerful attack possible, in terms of the capabilities of the underlying large language model used in the attack. Apart from this, the attack’s assumptions are the strongest possible: full knowledge of the learned parameters and architecture of the victim model, as well as the differentially private mechanism and its parameters. However, since we add fresh noise to each input exposed to the GPU, it is unlikely that a more powerful LLM will perform substantially better than ours.

- One possibility to improve the attack is to use the fact that adding fresh noise to the same quantity multiple times makes the process vulnerable to averaging attacks [3]. However, even though the augmented input X after each generated token contains the original tokens as well, they are not passed on to the GPU again as the multiplication result is kept in the KV cache. So an averaging attack cannot be launched this way. It may be possible to compare outputs after each layer of a transformer. But the input undergoes many non-linear transformations, and hence algebraically separating the noise from the original input to launch an averaging attack is not straightforward.

9 Conclusion

In this work, we show that splitting LLM inference between a trusted but slower TEE and an untrusted but faster GPU, with intermediate representations protected by differential privacy, enables the system to benefit from GPU acceleration while simultaneously protecting the prompt. We have shown that these representations can leak information about the original prompt, which motivates protecting the values sent outside the TEE. Our method does come with a trade-off. We show that uncontrolled differentially private noise can result in irreversible floating-point errors. However, unlike systems that use encryption to mask inputs, this error can be reined in by tuning the differential privacy parameter to balance privacy and accuracy of the LLM response. With the suggested values of the privacy parameter, we show that prompt reconstruction fails even with full knowledge of the LLM and the privacy mechanism. Beyond the use case considered in this paper, our method could also enable systems to leverage newly available, state-of-the-art GPUs without exposing information about potentially sensitive prompts to them.

References

- [1] Ahmad Al Badawi, Andreea Alexandru, Yuriy Polyakov, and Vinod Vaikuntanathan. Sok: Private llm inference using approximate homomorphic encryption. *Cryptology ePrint Archive*, 2026.
- [2] Davide Andreoletti, Alessandro Rudi, Emanuele Carpanzano, Francesco Lelli, and Tiziano Leidi. Privacy-preserving llm inference in practice: A comparative survey of techniques, trade-offs, and deployability. *Cryptology ePrint Archive*, 2026.
- [3] Hassan Jameel Asghar and Mohamed Ali Kaafar. Averaging attacks on bounded noise-based disclosure control algorithms. *Privacy Enhancing Technologies Symposium (PETS)*, 2020(2):358–378, 2020.
- [4] Sheldon Axler. *Linear algebra done right*. Springer, 2024.
- [5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [6] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. Intel tdx demystified: A top-down approach. *ACM Comput. Surv.*, 56(9), April 2024.
- [7] Md Hafizul Islam Chowdhury, Hao Zheng, and Fan Yao. Metaleak: Uncovering side channels in secure processor architectures exploiting metadata. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 693–707. IEEE, 2024.
- [8] Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free dolly: Introducing the world’s first truly open instruction-tuned llm, 2023.
- [9] Gobikrishna Dhanuskodi, Sudeshna Guha, Vidhya Krishnan, Aruna Manjunatha, Rob Nertney, Michael O’Connor, and Phil Rogers. Creating the First Confidential GPUs. *Commun. ACM*, 67(1):60–67, December 2023.
- [10] Jinshuo Dong, Aaron Roth, and Weijie J Su. Gaussian differential privacy. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 84(1):3–37, 2022.

- [11] Tian Dong, Yan Meng, Shaofeng Li, Guoxing Chen, Zhen Liu, and Haojin Zhu. Depth Gives a False Sense of Privacy: LLM Internal States Inversion. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 1629–1648, 2025.
- [12] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*, pages 265–284. Springer, 2006.
- [13] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and trends® in theoretical computer science*, 9(3-4):211–487, 2014.
- [14] Rusins Freivalds. Probabilistic machines can use less running time. In *IFIP congress*, volume 839, page 842, 1977.
- [15] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [16] Zhongshu Gu, Enriquillo Valdez, Salman Ahmed, Julian James Stephen, Michael Le, Hani Jamjoom, Shixuan Zhao, and Zhiqiang Lin. NVIDIA GPU Confidential Computing Demystified, July 2025.
- [17] Akshat Gupta, Atahan Ozdemir, Caoqinwei Gong, and Gopala Anumanchipalli. Geometric Interpretation of Layer Normalization and a Comparative Analysis with RMSNorm. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 375–407, Rabat, Morocco, March 2026.
- [18] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2 edition, 2002.
- [19] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [20] Jing Huang, Diyi Yang, and Christopher Potts. Demystifying verbatim memorization in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 10711–10732, 2024.
- [21] Timour Igamberdiev and Ivan Habernal. Dp-bart for privatized text rewriting under local differential privacy. In *Findings of the association for computational linguistics: ACL 2023*, pages 13914–13934, 2023.
- [22] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.
- [23] Jennifer King, Kevin Klyman, Emily Capstick, Tiffany Saade, and Victoria Hsieh. User privacy and large language models: an analysis of frontier developers’ privacy policies. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, volume 8, pages 1465–1477, 2025.
- [24] Haoran Li, Mingshi Xu, and Yangqiu Song. Sentence embedding leaks more information than you expect: Generative embedding inversion attack to recover the whole sentence. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 14022–14040, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [25] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.
- [26] Jämes Ménétrety, Christian Göttel, Anum Khurshid, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, and Shahid Raza. Attestation mechanisms for trusted execution environments demystified. In *IFIP International Conference on Distributed Applications and Interoperable Systems*, pages 95–113. Springer, 2022.

- [27] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander Rush. Text embeddings reveal (almost) as much as text. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12448–12460, Singapore, December 2023. Association for Computational Linguistics.
- [28] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser, 2 edition, 2018.
- [29] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. Smooth sensitivity and sampling in private data analysis. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, pages 75–84, 2007.
- [30] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In Pierre Isabelle, Eugene Charniak, and Dekang Lin, editors, *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics.
- [31] Tianxiang Shen, Ji Qi, Jianyu Jiang, Xian Wang, Siyuan Wen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Xiapu Luo, Fengwei Zhang, and Heming Cui. {SOTER}: Guarding Black-box Inference for General Neural Networks at the Edge. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 723–738, 2022.
- [32] Congzheng Song and Ananth Raghunathan. Information leakage in embedding models. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS ’20*, page 377–390, New York, NY, USA, 2020. Association for Computing Machinery.
- [33] Gemma Team, Aishwarya Kamath, Johan Ferret, et al. Gemma 3 technical report, 2025.
- [34] Florian Tramer and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In *International Conference on Learning Representations*, 2019.
- [35] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction {APIs}. In *25th USENIX security symposium (USENIX Security 16)*, pages 601–618, 2016.
- [36] Martin Unterguggenberger, Lukas Lamster, David Schrammel, Martin Schwarzl, and Stefan Mangard. Tme-box: Scalable in-process isolation through intel tme-mk memory encryption. In *Proceedings 2025 Network and Distributed System Security Symposium, NDSS 2025*. Internet Society, 2025.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [38] Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.
- [39] Jiaqi Xue, Yifei Zhao, Mengxin Zheng, Fan Yao, Yan Solihin, and Qian Lou. Securing Transformer-based AI Execution via Unified TEEs and Crypto-protected Accelerators, July 2025.
- [40] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [41] Ziqi Zhang, Chen Gong, Yifeng Cai, Yuanyuan Yuan, Bingyan Liu, Ding Li, Yao Guo, and Xiangqun Chen. No Privacy Left Outside: On the (In-)Security of TEE-Shielded DNN Partition for On-Device ML. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3327–3345, May 2024.

A Ethical Considerations

This study uses the Pile and Databricks Dolly datasets, both of which are widely used for language-model research. Our work focuses on reducing privacy risks during LLM inference by limiting the information exposed to an untrusted accelerator. The reconstruction experiments are conducted only to evaluate potential leakage from intermediate representations and to measure the effectiveness of the proposed protection mechanism.

B Example Prompts and Generated Responses

Table 7 provides additional examples comparing responses from the GPU baseline, EP1, EPN, and Slalom. Differences from the GPU baseline are highlighted in red.

C Prompt Reconstruction example

Table 8 shows the original prompt, the noise-free and noise-aware reconstructions, and their GPT-5.4 restored versions. The restored outputs improve coherence through minimal correction without adding new information.

Table 7: Additional example prompts and generated responses for the GPU baseline, EPN, and Slalom. Each response is limited to 200 generated tokens and therefore ends at the generation cap. Differences from the GPU baseline are shown in **red** for both EPN and Slalom.

Prompt	Given a reference text about Run Towards the Danger, tell me how many essays are part of the collection. <i>Context:</i> Run Towards the Danger is a 2022 Canadian essay collection by Sarah Polley, a former child star, director, and screenwriter. The six essays in the collection examine aspects of Polley’s career on stage, screen, and on film detailing her roles in a Stratford Festival production of Alice Through the Looking Glass, as well as her breakout roles in The Adventures of Baron Munchausen and the TV series Road to Avonlea. The book also revealed for the first time that Polley had been a victim of Jian Ghomeshi who sexually and physically assaulted her when she was 16 and he was 28.
GPU	There are 6 essays in the collection “Run Towards the Danger” by Sarah Polley. However, I do not have information about the specific titles of the essays. If you need more information about the essays, I suggest checking the book itself or a reliable online source. It’s worth noting that the book is not just a collection of essays about her career, but also a personal account of her experiences with Jian Ghomeshi, which may be a significant part of the book. If you’re looking for information about the essays, I recommend checking the book’s table of contents or a review. If you need more information about the book or its contents, I can try to help you find it. Just let me know! Also, I can tell you that the book has received critical acclaim for its powerful and thought-provoking exploration of Polley’s experiences with Ghomeshi and her career. Many reviewers have praised the book for its honesty, vulnerability, and insight
EP1	There are 6 essays in the collection “Run Towards the Danger” by Sarah Polley. However, I do not have information about the specific titles of the essays. If you need more information about the essays, I suggest checking the book itself or a reliable online source. It’s worth noting that the book is not just a collection of essays about her career, but also a personal account of her experiences with Jian Ghomeshi, which may be a significant part of the book. If you’re looking for information about the essays, I recommend checking the book’s table of contents or a review. If you need more information about the book or its contents, I can try to help you find it. Just let me know! Also, I can tell you that the book has received critical acclaim for its powerful and thought-provoking exploration of Polley’s experiences with Ghomeshi and her career. Many reviewers have praised the book for its honesty, vulnerability, and insight
EPN	There are 6 essays in the collection “Run Towards the Danger” by Sarah Polley. However, I do not have information about the specific titles of the essays. If you need more information about the essays, I suggest checking the book itself or a reliable online source. It’s also worth noting that the book is not just a collection of essays about her career, but also a personal account of her experiences with Jian Ghomeshi, which may be a significant part of the book. If you’re looking for information about the essays, I recommend checking the book’s table of contents or a review. If you need more information about the book or its contents, I can try to help you find it. Just let me know! Also, I can tell you that the book has received critical acclaim for its powerful and thought-provoking exploration of Polley’s experiences with Ghomeshi and her journey towards healing and recovery. If you’re interested in learning more about the
Slalom	There are 6 essays in the collection “Run Towards the Danger” by Sarah Polley. However, I do not have information about the content of the essays. The information provided only mentions that the essays examine aspects of Polley’s career and a personal experience of being a victim of Jian Ghomeshi. If you need more information about the essays, I suggest checking the book itself or a reliable online source. However, I can tell you that the book is a memoir and a collection of essays, not a collection of standalone essays. It’s a personal and reflective work that explores Polley’s experiences and career, including her experiences with Ghomeshi. If you’re looking for a specific type of essay, I’d be happy to try and help you find more information. If you need more information about the essays, I suggest checking the book itself or a reliable online source. The book has received positive reviews for its candid and thought-provoking exploration of Polley’s

Prompt	Extract the ingredients in bee pollen from the text. <i>Context:</i> Bee pollen, also known as bee bread and ambrosia, is a ball or pellet of field-gathered flower pollen packed by worker honeybees, and used as the primary food source for the hive. It consists of simple sugars, protein, minerals and vitamins, fatty acids, and a small percentage of other components. Bee pollen is stored in brood cells, mixed with saliva, and sealed with a drop of honey. Bee pollen is harvested as food for humans and marketed as having various, but yet unproven, health benefits.
Reconstructed Prompt (noise-free)	Extract ingredients in bee pollen from the text. Context: B pollen also as bread ambia is ball pellet fieldather flower packed workerbe, used the food for hive It of sugars protein, a or of ged pollen by honeyes and with, acids and, a percentage other. pollen stored bro cells mixed saliva, a as primary source the. consists simple, with, minerals vitamins fatty, acids as various but unven health. benefits. pollenstored bro cells combined saliva, as small of components Bee is as for humans marketed having, sealed a of components Beer is inod, with saliva and with drop honey Bee, benefits
Restored Prompt (noise-free) - GPT-5.4	Extract ingredients in bee pollen from the text. Context: Bee pollen, also known as bee bread and ambrosia, is ball or pellet field-gathered flower pollen packed by worker bees, used as the food source for the hive. It consists of simple sugars, protein, a large percentage of gathered pollen by honeybees, mixed with nectar, fatty acids, and minerals, and a small percentage of other components. Bee pollen is stored in brood cells, combined with saliva, and sealed with a drop of honey. Bee pollen is used as food for humans and marketed as having various but unproven health benefits.
Reconstructed Prompt (noise-aware, $\epsilon = 1$)	The of the is a of the, and it's not just because they both. Both are in their own way to be loved for who they, but also have been by others for reasons that may or not. of the has always been a bit different from other. They're more independent than some, but still need love care. of on the hand very much people around them, needs support and love. both of these qualities make them special, but can also lead problems if not handled right. of is often seen as the perfect partner, someone who will stand you through thick and thin, never leave your side, and always there when needed. However, this can sometimes lead to dependency, where one becomes too reliant another, losing independence and autonomy. of, on the other hand, is often portrayed as the bad boy, someone with attitude and who doesn't play games. However can also be hurtful, leaving person feeling rejected abandoned. of all, however, is the most complicated. They so much like each other, yet at same time don't want to lose individuality. This can lead to conflicts, where try to balance their relationship with their own needs desires. of is especially difficult, as they so willing help each other out, but at same
Restored Prompt (noise-aware, $\epsilon = 1$) - GPT-5.4	The is a of the, and it's not just because they both. Both are, in their own way, to be loved for who they are, but also have been by others for reasons that may or may not. This has always been a bit different from the others. They're more independent than some, but still need love and care. On the other hand, very much needs the people around them, and needs support and love. Both of these qualities make them special, but can also lead to problems if not handled right. This is often seen as the perfect partner, someone who will stand by you through thick and thin, never leave your side, and always be there when needed. However, this can sometimes lead to dependency, where one person becomes too reliant on another, losing their independence and autonomy. , on the other hand, is often portrayed as the bad boy, someone with attitude and confidence who doesn't play games. However, this can also be hurtful, leaving the other person feeling rejected and abandoned. Most of all, however, is the most complicated. They are so much like each other, yet at the same time don't want to lose their individuality. This can lead to conflicts, where they try to balance their relationship with their own needs and

Table 8: Example of the original prompt, its QKV-only Layer 0 reconstructions under noise-free and noise-aware conditions, and the corresponding GPT-5.4 restored outputs.